COMP710: Game Programming

Studio Session 05 – Exercises 05a: Kien Tran

Overview

- Video Game Structure
- Real-Time Simulation: The Game Loop, Computer Graphics…
- Game Engine Concepts:
  - Resource Management
  - Build Targets
- The COMP710 “GP Framework”:
  - Project Configuration… Build Targets… Project File Structure…
  - Logging, The **`Game`** Object, The **`main`** Function, The **`Renderer`** Interface
  - **`Texture`**, **`TextureManager`**, **`Shader`**, GLSL, **`Matrix4`**, **`VertexArray`**
  - Sprites!

Video Game Structure…

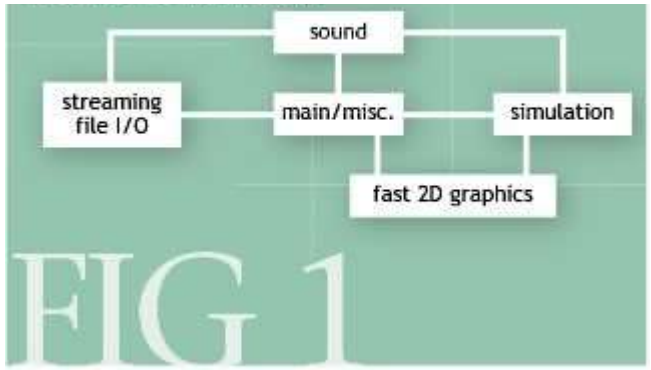
Complexity and scale has risen over time…

Video Game Structure

- Modules:
  - Different areas of responsibilities…
  - Each contains particular kinds of algorithms…

- Let's compare game complexity, over time:
  - 1) Early 2D games…
  - 2) Early 3D games…
  - 3) Modern 3D games…
  - 4) Modern 3D massively multiplayer games…

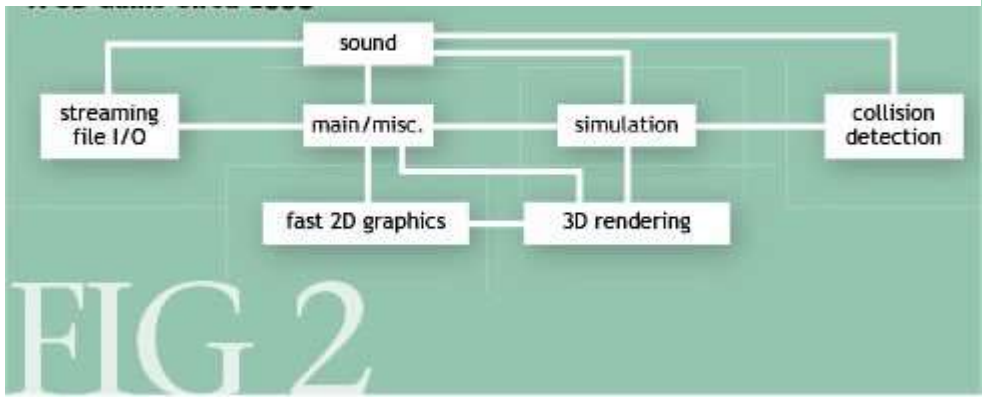
1) A 2D Game circa 1994:



http://queue.acm.org/detail.cfm?id=971590

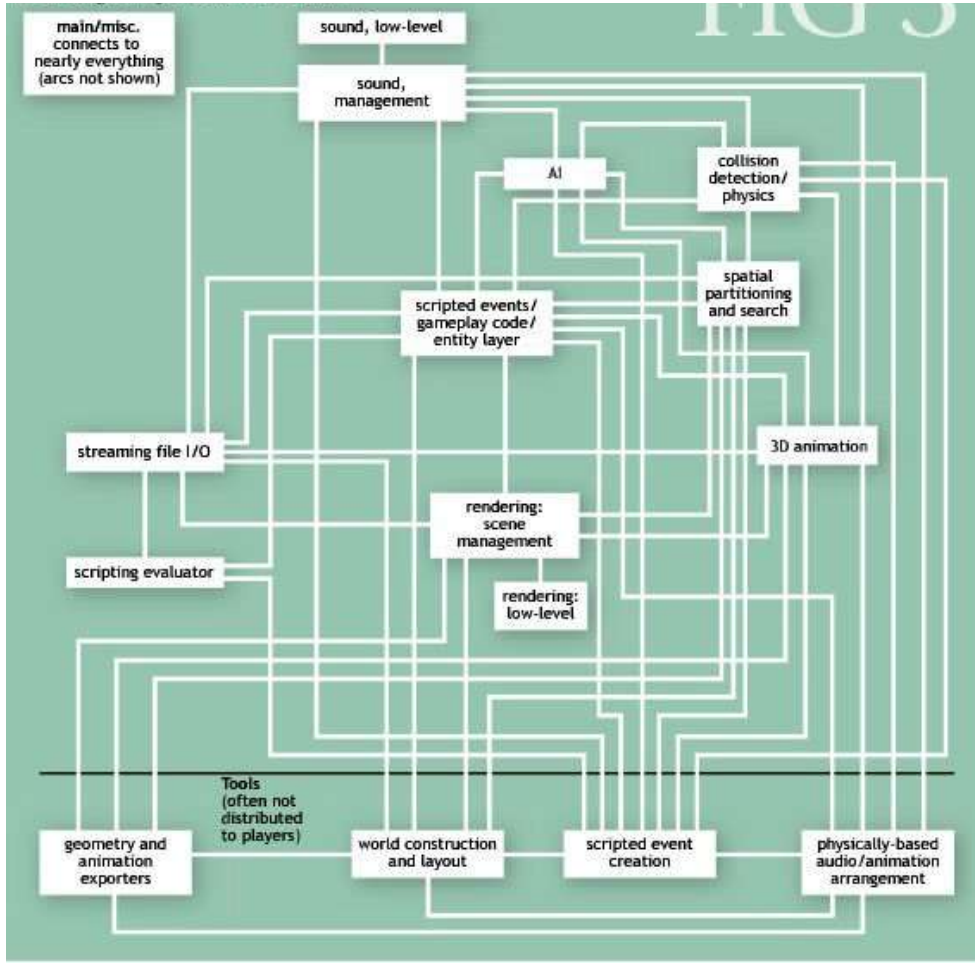
Video Game Structure

2) A 3D Game circa 1996:



http://queue.acm.org/detail.cfm?id=971590

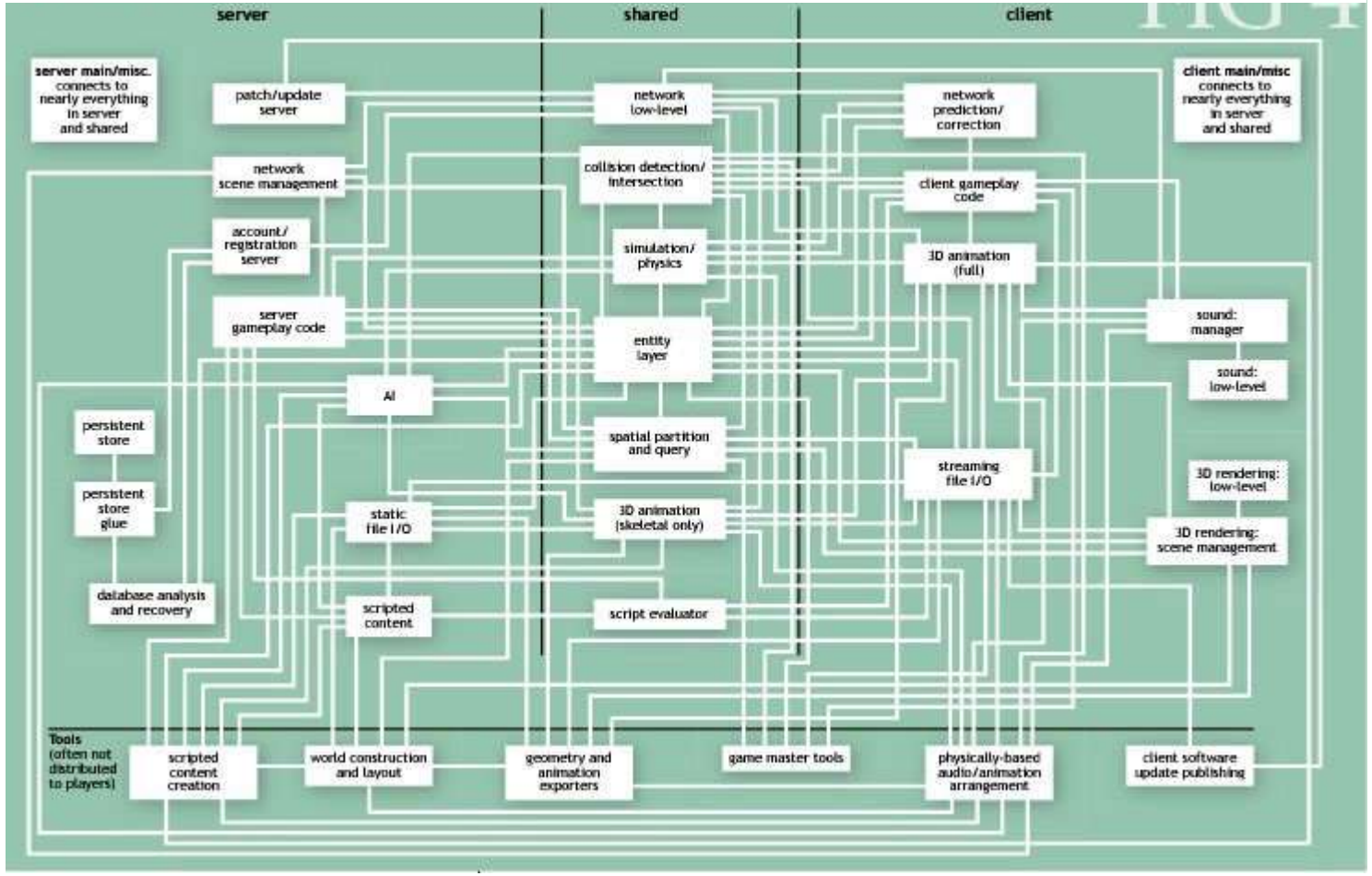
3) A 3D Single Player Game circa 2004:



http://queue.acm.org/detail.cfm?id=971590

Video Game Structure

4) A 3D Massively Multiplayer Game circa 2004:



http://queue.acm.org/detail.cfm?id=971590

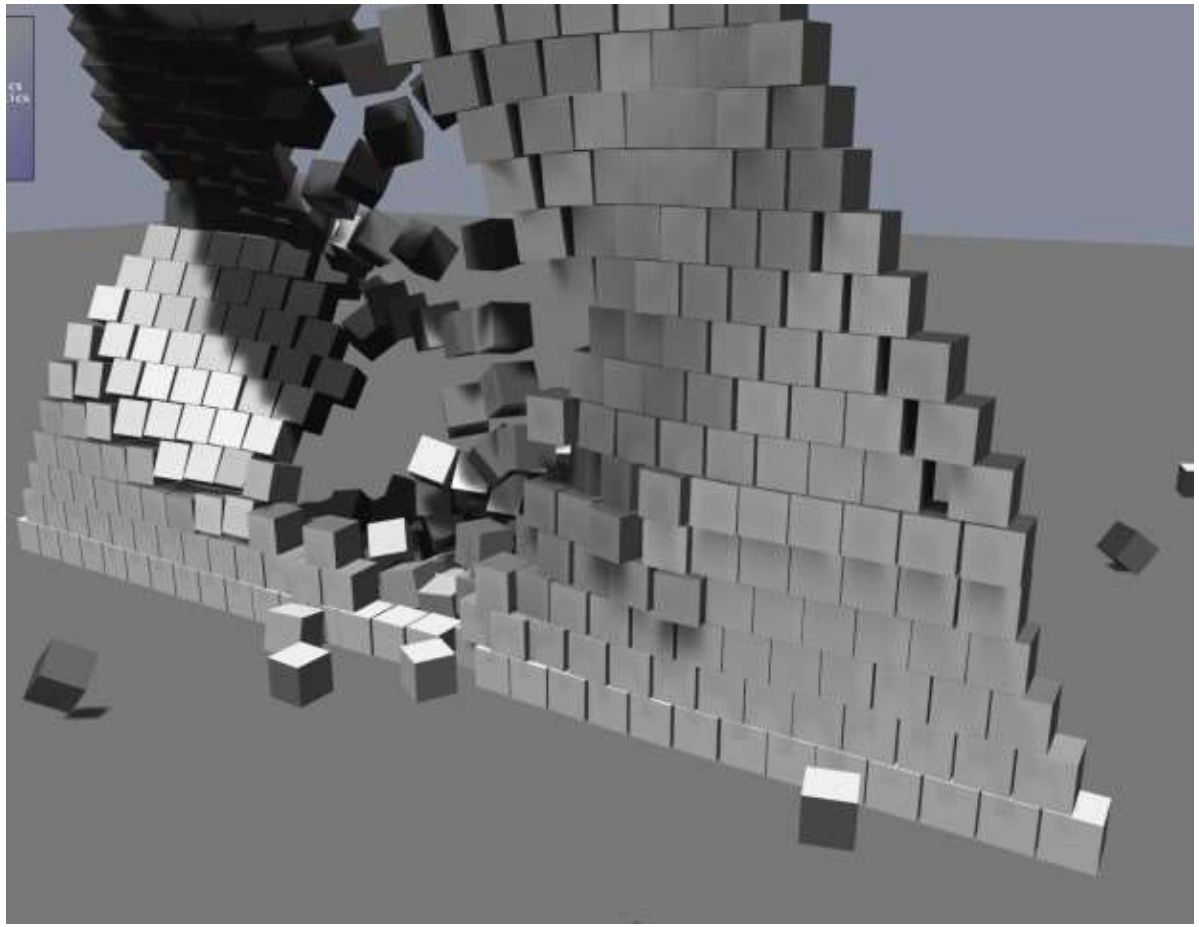
Video Game Structure: Modules

- Graphics / Rendering Engine:
  - Scene Management
  - Models
  - Bones
  - Animation
  - Shaders
  - Lighting… Materials
  - Shadows
  - Camera Controls
  - Particles
  - Workflow tools… exporting/converting assets…

http://images.bit-tech.net/content_images/2009/03/how-ai-in-games-works/totalwar.jpg

Video Game Structure: Modules

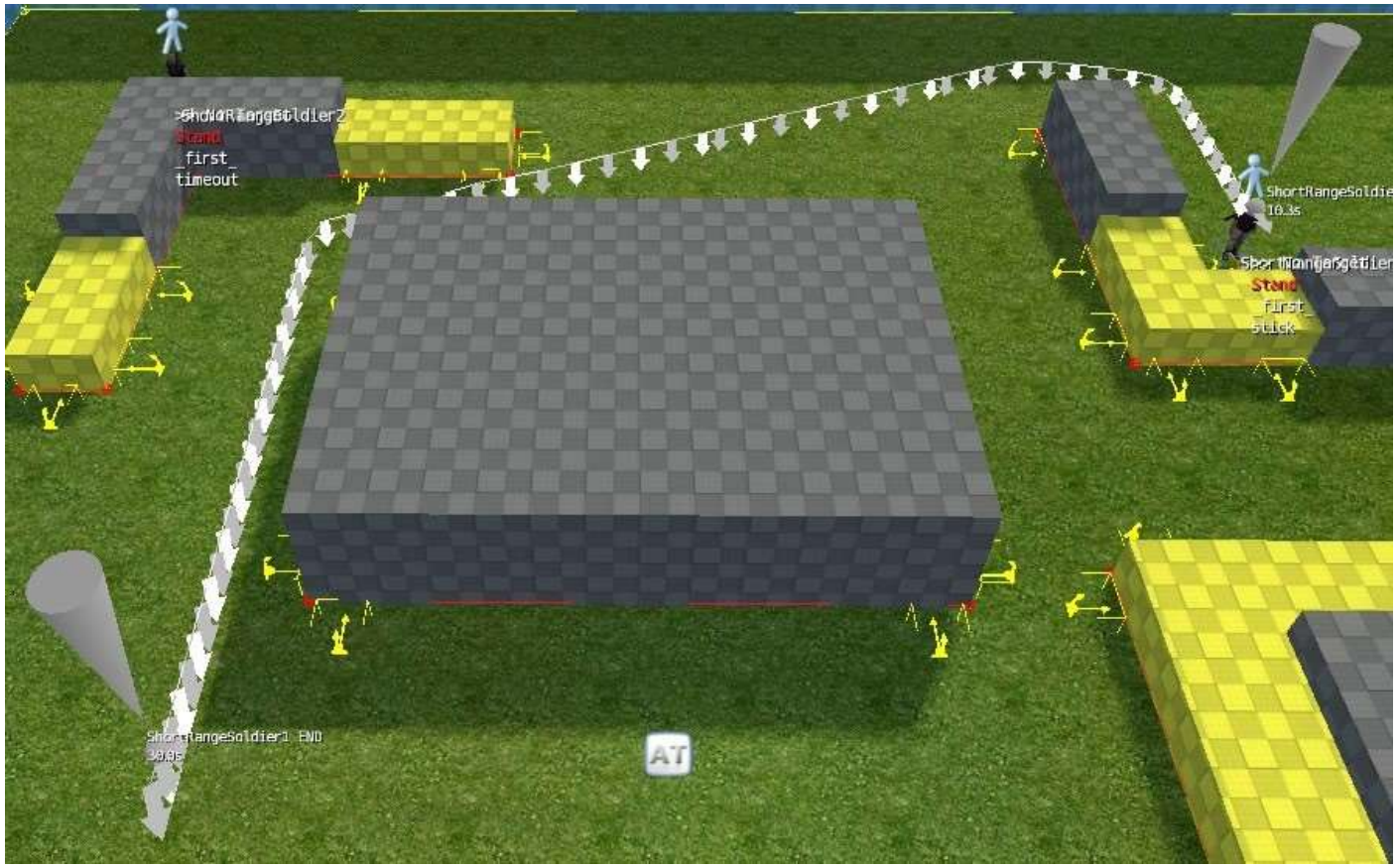
- Physics Engine:
  - Collision Detection
  - Collision Response
  - Kinematic Simulation
  - Rigid Body Dynamics:
    - Joints, Sliders, Ball, Hinge, …
  - Mass/Spring
  - Vehicles…
  - Ragdoll…
  - Particles:
    - Fluids, Smoke, Cloth, …



http://www.havok.com/sites/default/files/styles/galleryformatter_slide/public/Image27.png?itok=vIRCsTZX

Video Game Structure: Modules

- Artificial Intelligence:
  - Behaviour
  - Strategy
  - Learning
  - Path Planning / Finding
  - World/Perception
  - Team collaboration…
- Scripting:
  - Data Driven



https://docs.cryengine.com/download/attachments/1048664/DebugView_3D_TimeRangeInfo.jpg?version=1&modificationDate=1273062839000&api=v2

Video Game Structure: Modules

- Networking:
  - Multiplayer
  - Client/Server
  - Encryption
  - Synchronisation
  - Lobby and User Management
- Streaming:
  - Video, Audio
  - Level Data
- Events:
  - Event Driven Model

https://giantbomb1.cbsistatic.com/uploads/original/0/8485/1199989-left4dead2_2009_11_27_01_45_37_68.jpg

Video Game Structure: Modules

- Audio Engine:
  - Play Sound Effects
  - Play Music
  - 3D Sound
  - Occlusion
  - Effects: Echo, Reverb, etc…
  - Streaming…
- Memory Management…
- Process Management…

http://gameaudiofactory.com/wp-content/uploads/2012/05/studio.jpg

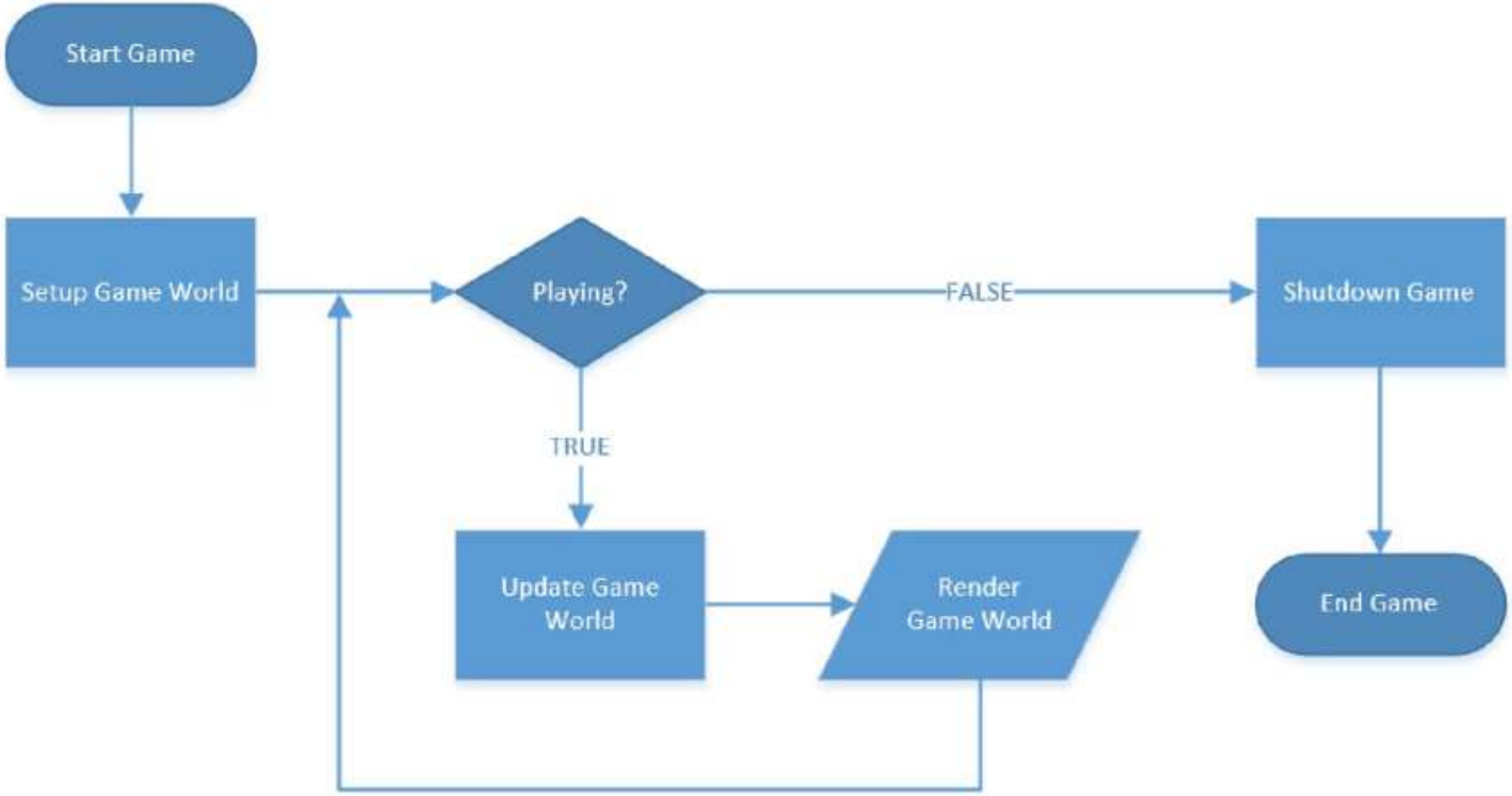
Real-Time Simulation…

The game loop…

Real-Time Simulation: The Game Loop

- Interactive program: A Game!
- Decoupling the progression of time from processor speed…
- The game loop processes input…
  - Not blocking!
- Processing moves the simulation forward in time.
  - By some amount of time…
- Drawing renders the game.

Real-Time Simulation: The Game Loop



Real-Time Simulation: The Game Loop

- High-level pseudo code example:

```
Initialisation();    // Setup the game world.

while (playing)      // The real-time loop.
{
    GetInput();      // Retrieve input.
    Process(dt);     // Moves the sim forward in time.
    Draw();          // Renders the updated sim.
}

Shutdown();          // Release resources.
```

Real-Time Simulation: The Game Loop

- The Game Loop continued…
  - Frames per Second (FPS):
    - The number of frames rendered, per real world second.
    - Common targets: 30 FPS or 60 FPS… Higher FPS requirements needed for VR games!
  - If the game loop iterates too slowly…
    - The game becomes a slide show… 1 FPS or 2 FPS… ☹
  - If the FPS is high, then… smooth gameplay! ☺
  - What makes the “frame rate” high/low?
    - How much “work” is done per frame… The algorithms in the game logic!
    - Speed of the target platform… CPU, GPU…

Real-Time Simulation: The Game Loop

- The Game Loop continued…
  - Played an old game on a newer PC?
    - Old games may speed up on new hardware! ☹
      - Making it impossible to play! ☹
    - Old games: Fixed hardware targets… "TURBO" button?
    - **Solution: Run the game at a consistent speed!**
  - Target FPS?
    - 60 FPS: means 60 frames rendered per second…
    - Time per frame: 1/60 = 0.01666 seconds (16 milliseconds per frame)
    - If all the game's updating, and rendering can be done in less than 16 milliseconds…
      - Then the frame rate can be at least 60 FPS.
    - So… maybe slow the game down… possibly… sleep the process…

http://4.bp.blogspot.com/_6z5UPWubGhA/TKyqiB5q71I/AAAAAAAA8c/VWIeBWl9spY/s320/turbo_button.jpg

The Game Loop: Timing

- A 60 FPS game loop; pseudo code example:

```
while (playing)
{
    float s = getCurrentTime(); // Seconds.

    Process();
    Draw();

    Sleep(s + 0.016666 - getCurrentTime());
}
```

The Game Loop: Timing

- What if the loop iteration takes longer than 0.016666 seconds?
  - Do less “work” per frame!
    - Cut down on the algorithms…
      - Remove fancy graphics rendering…
      - Remove fancy physics calculations…
      - Remove fancy artificial intelligence calculations…
        - The poor game! ☹ Where is the fun? ☹
  - A solution: **Time steps**
    - Variable time step: Time step based upon how much real world time passed since the last frame. **The longer the time, the bigger the time step.**
  - Then update the simulation based upon the amount of time between frames:
    - **Delta Time**. The amount of time that elapsed between frames.
      - But beware…. non-deterministic… can cause instability in game systems such as: physics simulation… networking communication… artificial intelligence processing…

The Game Loop: Timing

- The variable time step game loop; pseudo code example:

```
float lastTime = getCurrentTime();

while (playing)
{
    float current = getCurrentTime();
    float delta = current - lastTime;
    Process(delta);
    Draw();

    lastTime = current;
}
```

The Game Loop: Processing / Updating

- Updating based upon time…
  - The change in time… delta time… (or **dt** for short…)
- So… game entities need to be **updated with respect to time**…
  - Position (e.g.: a 2D coordinate)
  - Velocity (e.g.: 2D vector – some units per second).
  - Change the position per frame by the velocity… and the velocity vector is scaled by time! ☺

```
void
Enemy::Process(float dt)
{
    m_position += (m_velocity * dt);
}
```

The Game Loop: Timing

- Better Solution: Fixed Time step
  - Stable for physics simulation…
  - Series of fixed time steps…
  - At the start of a frame:
    - Calculate the “lag”: how much real time passed…
  - Inner loop inside the game loop to update the game world entities…
    - One fixed step, until caught up to the “lag”…
  - Once caught up, draw the game again!

The Game Loop: Timing

- The fixed time step game loop; pseudo code example:

```
float lastTime = getCurrentTime(); // in seconds...
float lag = 0.0f;

while (playing)
{
    float currentTime = getCurrentTime();
    float delta = currentTime - lastTime;
    lastTime = currentTime;
    lag += delta;
    ProcessInput();
    while (lag >= 0.016666)
    {
        Process(0.016666);
        lag -= 0.016666;
    }
    Draw();
}
```

The Game Loop: Processing / Updating

- "Game Programming Design Pattern" The *Update* Method:
  - **`Process(dt)`** or **`Update(dt)`**
  - A chance to move the real-time simulation forward in time…
    - In this method a game object can simulate its behaviour…
  - The method will updates with respect to time…
    - How much time has elapsed between process calls (the parameter holds this value)?
    - Running the simulation takes time… (processing the game world…)
    - Rendering the simulation takes time… (drawing the game world…)

Computer Graphics

Background Knowledge…

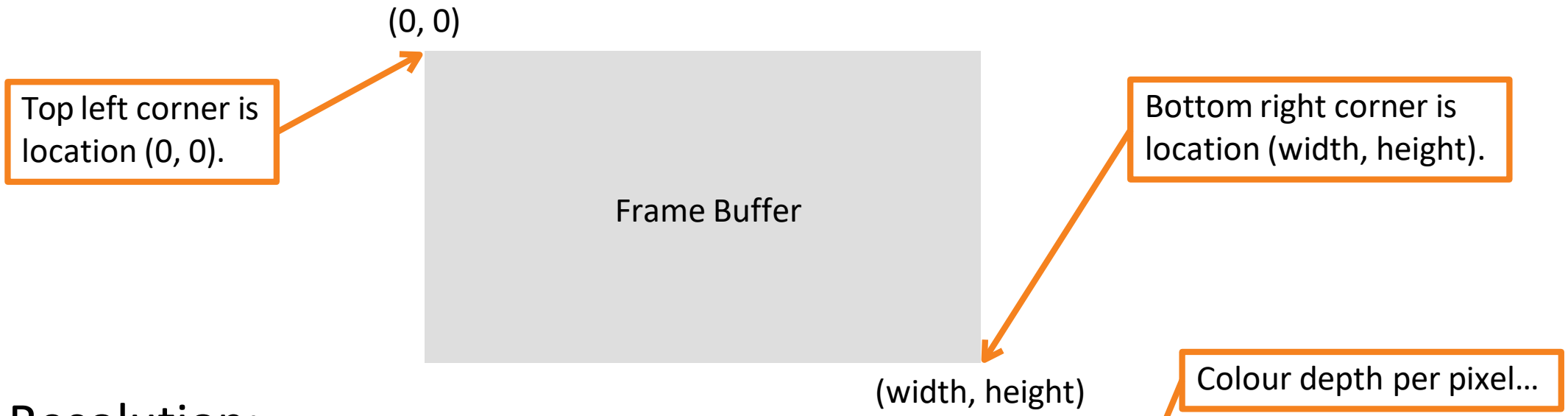
and 2D Graphics…

Computer Graphics

- Pixel: Picture Element
  - The smallest addressable visual element.
- Colour Format:
  - RGB: R8G8B8: 24 bit colour: 8 bits red, 8 bits green, 8 bits blue.
- The Frame Buffer: Array of pixels in RAM.
  - Size = (Width * Height * Colour Depth) bytes…
- Aspect Ratio:
  - Width : Height or Width/Height
- Refresh Rate:
  - 60Hz, 100Hz, etc…

2D Graphics: The Frame Buffer

- The Frame Buffer's Coordinates…

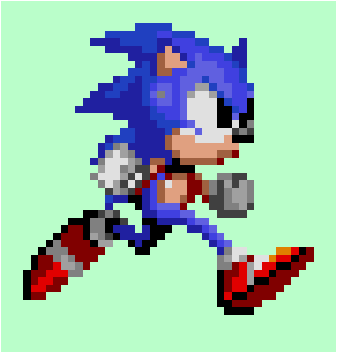


- Resolution:
  - Is computed as: (width * height * colour depth)
  - How much memory does this use? 800px * 600px * 24bits = 1.44 Megabytes

2D Graphics: Sprites

- Sprite: A two dimensional image…
  - Generally integrated into a larger scene…
- Digital art edited at the pixel level…
  - Many techniques: Line art, limited palette, dithering, hand-made anti-aliasing, etc…
- Types:
  - Isometric, Non-isometric…
    - Game examples: Sim City 2000 (Maxis, 1994)
- Possible Formats: BMP, GIF, PNG, JPG, TGA, …
  - Likely to be target platform specific!

Sonic The Hedgehog 2 (Sega, 1992):



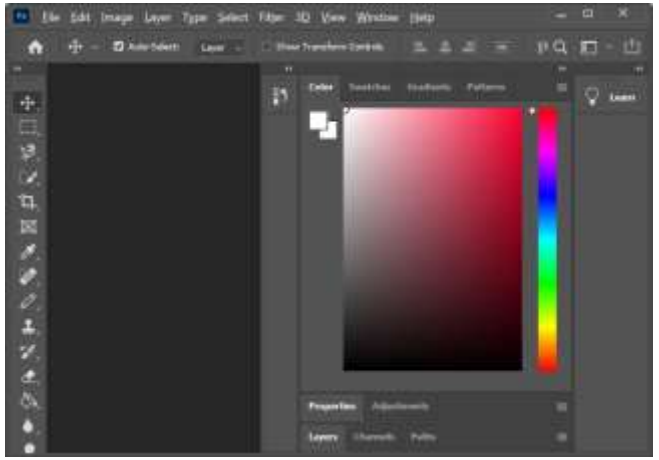
http://www.spriters-resource.com/fullview/10073/

http://piq.codeus.net/static/media/userpics/piq_30130_400x400.png

http://i245.photobucket.com/albums/gg62/witchyhoy3/2479.gif

2D Graphics: Sprites

- Game Artists: Ways to create sprites…
  - Raster generated:
    - Created pixel by pixel…
    - Possibly hand drawn first… then digitised…
  - Vector-based: Good for scaling…
  - 3D-based: Pre-rendered based upon 3D data…
    - Examples:
      - Donkey Kong Country (Rare, 1994) on the SNES.
      - Killer Instinct (Rare, 1994) on the SNES.
      - Diablo (Blizzard North, 1996) on Windows.



Adobe's **Photoshop**

Blastmode's **PixelBasher**
https://pixelbasher.dev/

2D Graphics: Alpha Transparency

- The “see through” part of the sprite…
  - Layering of sprites created by draw order…
- First steps… use the PNG format… ☺
  - Beware: The BMP format does not support alpha!
- Mask Layer…
  - Content creation tool usage: **Adobe Photoshop**
  - Generally signalled by a grey and white checkerboard…

Computer Graphics: Screen Tearing

https://camo.githubusercontent.com/b559ed2a3f00c1f79173839dc4988d7bb6c0a4ec33d30d874c6b706ff1c13034/68747470733a2f2f662e636c6f75642e6769746875622e636f6d2f6173736574732f333738393232362f3439323436372f343236306430343461d626162322d313165322d383731662d333831653634316333365383332e706e67

2D Graphics: Double Buffering

- Double Buffering, the use of two frame buffers…
  - **Front**: What is currently drawn to the screen.
  - **Back**: Area for the real-time rendering program to draw to.

  - Once the “back buffer” is completely drawn, it is promoted to become the “front buffer”:
    - Buffer Swapping:
      - **Blit**: Copying from the back to the front, byte by byte.
      - **Flip**: Pointer swapping, simply swap the pointers to the back and front buffers.

  - If swapping is synced with screen refreshing:
    - No tearing! ☺

2D Graphics: Painters Algorithm

- Draw Order: Painters algorithm…
  - High-level pseudo code example:

```
Clear();
DrawBackground();
DrawStars();
DrawEnemies();
DrawBullets();
DrawPlayer();
DrawExplosions();
DrawHeadsUpDisplay();
Present();
```

Game Engines…

Some core concept ideas behind game engines…

Game Engine: Core Concepts…

- Purpose: Reusable components across games… “Recyclability”
- Importance: Fit for purpose…
- Architecture: Abstraction, Modularity…
- Design: Purpose… Particular style of game?
- Data Pipelines: Asset tool chain, Export processes, Editors, Viewers…

Game Engine: Considerations…

- Development Pipeline:
  - Engine source code availability?
  - Live preview of executable on target platform…
  - Middleware integration…
- Adaptability:
  - Resource management flexibility?
- Support:
  - Access to development builds…
  - Developer support, forums, etc…

Game Engine: Broad Categories…

- “High-fidelity”:
  - Unreal Engine (Epic Games), CryEngine (Crytek), Source (Valve), idTech (id Software)
- “Mid-range”:
  - Vision (Havok), Gamebryo (Emergent), Vicious Engine (Vicious Cycle Software), BlitzTech (Blitz Games Studios)
- “Casual”:
  - Unity (Unity Technologies), Torque 3D (GarageGames), ShiVa (Stonetrip), Marmalade (Marmalade Technologies)

In COMP710 we are going to…

…build our own reusable framework (or “engine” ☺)

COMP710 “GP Framework” – The Core…

- Simple DirectMedia Layer (**SDL**)
  - Cross-platform development library designed to provide low-level access to audio, keyboard, mouse, joystick, and graphics hardware via OpenGL and Direct3D.
  - Used by video playback software, emulators, and popular games including Valve’s award winning catalogue and many Humble Bundle games.
  - Written in C, works natively with C++.

- OpenGL Extension Wrangler Library (**GLEW**)
  - Cross-platform open-source C/C++ extension loading library…
  - Efficient run-time mechanisms for determining which OpenGL extensions are supported on the target platform…

COMP710 “GP Framework” – The Core…

- SDL2: Version: 2.0.22 (**`https://www.libsdl.org/`**)
  - SDL2_image – Version: 2.6.1 (`https://github.com/libsdl-org/SDL_image/releases`)
- GLEW: Version: 2.1.0 (**`http://glew.sourceforge.net/`**)
- **`Studio5Resources.zip`** on Canvas:
  - **`Studio5Resources\SDL2-2.0.22\*`**
  - **`Studio5Resources\SDL2_image-2.6.1\*`**
  - **`Studio5Resources\glew-2.1.0\*`**
  - These contain folders: **`include`** (with **`.h`** files) and **`lib`** (with **`.lib`** files)

COMP710 “GP Framework” – The Core…

- Simple DirectMedia Layer: Version 2.0.22

Get the current **stable** SDL version 2.0.8

SDL Documentation:
`https://wiki.libsdl.org/`

SDL
Simple Directmedia Layer

Main
- About
- Bugs
- Licensing
- Credits
- Feedback

Documentation
- Wiki
- Forums
- Mailing Lists

Download
- SDL 2.0
- SDL 1.2
- SDL Mercurial
- Bindings

SDL version 2.0.8 (stable)

Source Code:

SDL2-2.0.8.zip - GPG signed
SDL2-2.0.8.tar.gz - GPG signed

Runtime Binaries:

Windows:
SDL2-2.0.8-win32-x86.zip (32-bit Windows)
SDL2-2.0.8-win32-x64.zip (64-bit Windows)

Mac OS X:
SDL2-2.0.8.dmg

Linux:
Please contact your distribution maintainer for updates.

GPG Signature:

The source code to this release has been signed by Sam Lantinga.
You can get the public key from any keyserver with the key id 0xA7763BE6, or directly from Sam's home page: slouken-pubkey.asc
The public key fingerprint should be:

```
pub  1024D/A7763866 2001-01-05 Sam Lantinga <slouken@libsdl.org>

     Key fingerprint = 1528 635D 8053 A57F 7701  E086 38A5 9377 A776 3866
```

For more information about public key signatures, see http://www.gnupg.org/

COMP710 “GP Framework” – The Core…

- The OpenGL Extension Wrangler (GLEW): Version 2.1.0

Latest Release: 2.1.0

Download
Usage
Building
Installation
Source Generation
Change Log

GitHub
Issues
Pull Requests
Authors
Licensing

SourceForge Page

Last Update: 07-31-17

OpenGL
GitHub
Travis CI
SOURCEFORGE

The OpenGL Extension Wrangler Library

The OpenGL Extension Wrangler Library (GLEW) is a cross-platform open-source C/C++ extension loading library. GLEW provides efficient run-time mechanisms for determining which OpenGL extensions are supported on the target platform. OpenGL core and extension functionality is exposed in a single header file. GLEW has been tested on a variety of operating systems, including Windows, Linux, Mac OS X, FreeBSD, Irix, and Solaris.

Downloads

GLEW is distributed as source and precompiled binaries.
The latest release is 2.1.0[07-31-17]:

Source	ZIP | TGZ
Binaries	Windows 32-bit and 64-bit

An up-to-date copy is also available using git:

- github
  `git clone https://github.com/nigels-com/glew.git glew`

Supported Extensions

The latest release contains support for OpenGL 4.6, compatibility and forward-compatible contexts and the following extensions:

- OpenGL extensions
- WGL extensions
- GLX extensions

News

- [07-31-17] GLEW 2.1.0 adds support for OpenGL 4.6, new extensions and minor bug fixes
- [07-24-16] GLEW 2.0.0 adds support for forward-compatible contexts, adds new extensions, OSMesa and EGL support, MX discontinued and minor bug fixes
- [08-10-15] GLEW 1.13.0 adds support for new extensions, fixes minor bugs

Let’s start by creating a new Solution and Project for this…

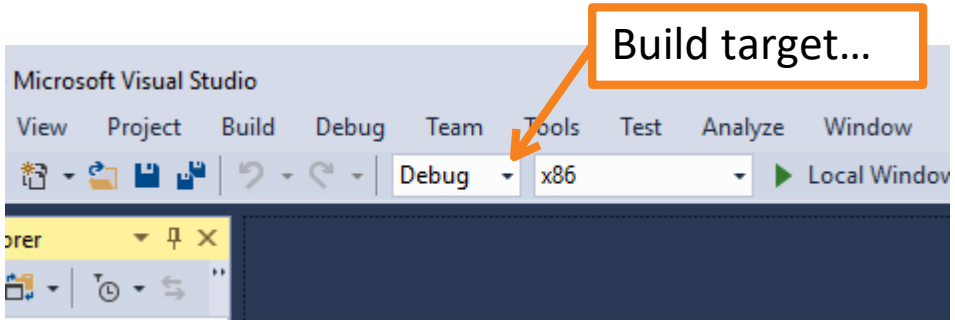
Using Visual Studio…

Build Targets…

Debug build versus Release build…

Build Targets: Debug versus Release

- Projects have separate **Debug** and **Release** configurations…
- Build the **Debug** version for debugging purposes…
  - Allows the debugging features to work… ☺
    - Assert, memory watching, memory leak detection, memory trampling detection, useful runtime information, warnings, and errors, etc…
- Build the **Release** version for final release distribution…
  - This is the executable that is shipped to the customer. It's faster… ☺
- A project should always be buildable and runnable in both targets!



Configuring the project… these need to be set up with relative paths… ☺

Do not use absolute paths! ☹

Advice…
When you start a new game project…

Choose a “codename”

Game Production: Project Configuration

- Pick a “gamename”… something unique to identify your game project… this becomes the root directory of your project…
  - This does not need to be the final name of the game!
- Configuration Management…
  - The directory structure inside your “gamename” folder:

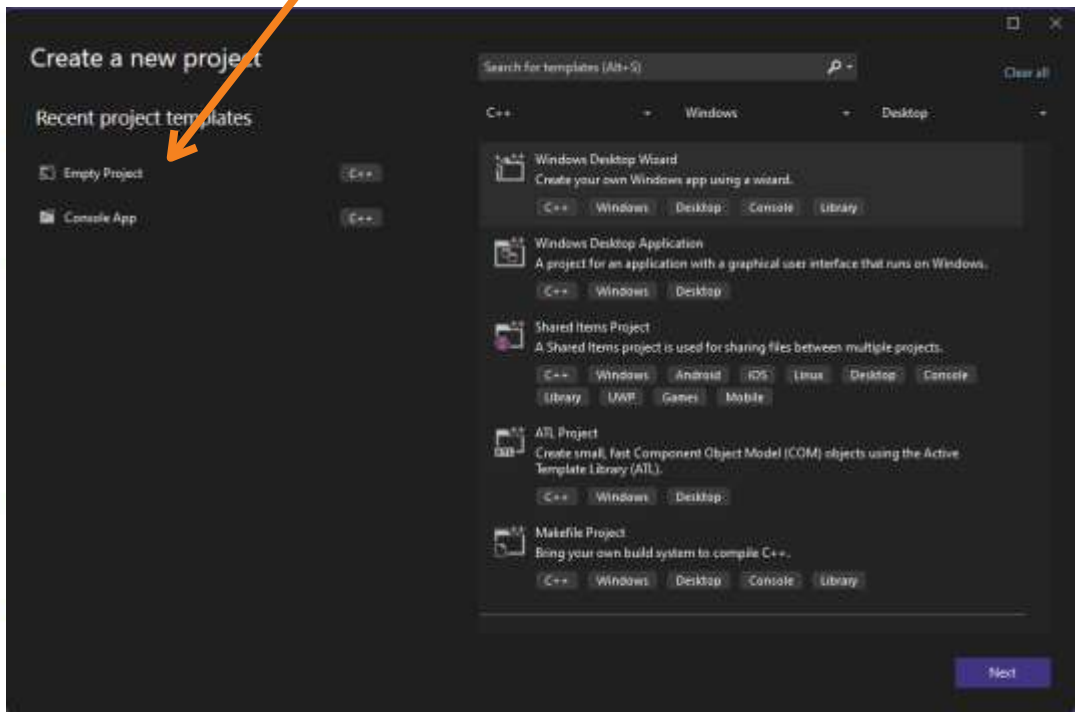
    + **`docs`**: References for the development team…
    + **`assets`**: Raw assets, art, animation, sound, …
    + **`temp`**: Target build output files… separate directories…
    + **`lib`**: Third-party libraries…
    + **`game`**: Source code,Release Build, all game data, hierarchy…
    + **`test`**: Testing files, scripts, cheat unlocks, test utilities…
    - Project Solution/Makefile,

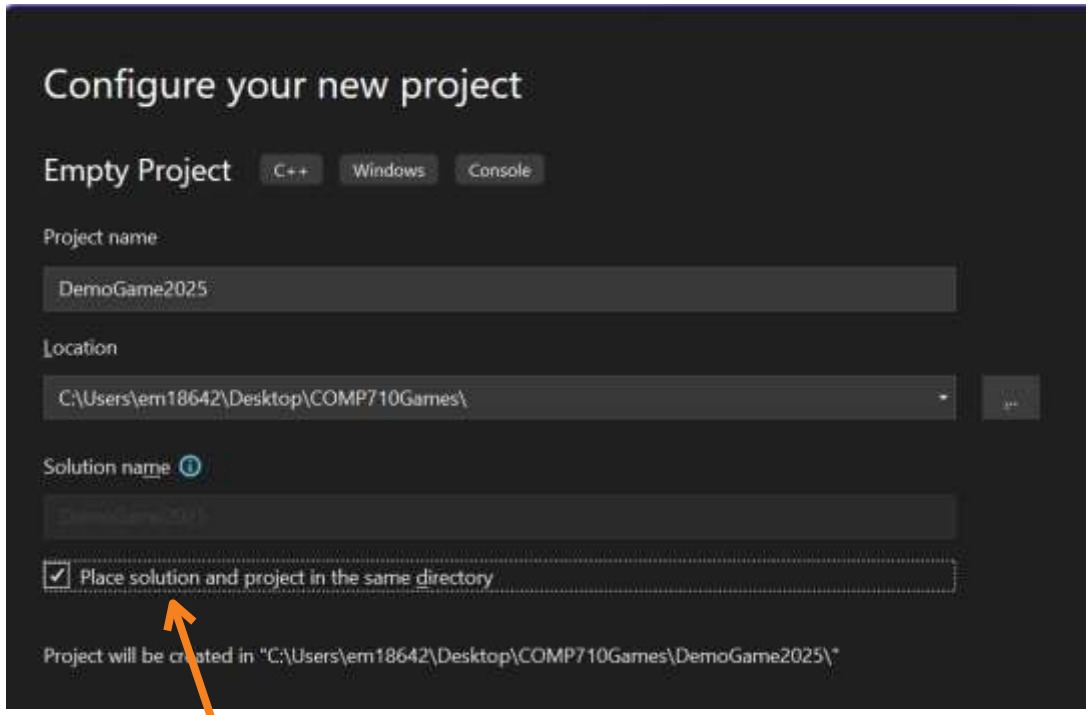
Game Production: Project Configuration

- The “game” folder:
  - Contains all the assets required to run the game…
    - Compiled executable…
    - Game-ready assets…
  - This folder can be zipped and distributed…
    - It contains everything a player (or reviewer, or marketing department) need to run the game…
    - And does not contain any source, or intermediate data that goes into crafting the game…
  - An installer can be generated based upon the contents of this folder…

Game Production: New project

Create an empty project





Select this option

Game Production: Setup

- New game project will create a folder named “gamename” and Inside this should be the **`.sln, .vcxproj,`**…
- Create sub-folders following given directory structure
- Copy the following folders from **`Studio5Resources`** to **`gamename\lib`**
  - **`SDL2-2.0.22\include\*,`**
  - **`SDL2-2.0.22\lib\*,`**
  - **`SDL2_image-2.6.1\include\*,`**
  - **`SDL2_image-2.6.1\lib\*,`**
  - **`glew-2.1.0\include\*,`**
  - **`glew-2.1.0\lib\*,`**
- Copy sprite files in **`Studio5Resources\Sprites`** to **`gamename\assets`**
- Copy DLL files
  - Copy: **`Studio5Resources\SDL2-2.0.22\dll\SDL2.dll`** to **`gamename\game\SDL2.dll`**
  - Copy: **`Studio5Resources\SDL2_image-2.6.1\dll\SDL2_image.dll`** to **`gamename\ game\SDL2_image.dll`**
  - Copy: **`Studio5Resources\glew-2.1.0\dll\glew32.dll`** to **`gamename\game\glew32.dll`**

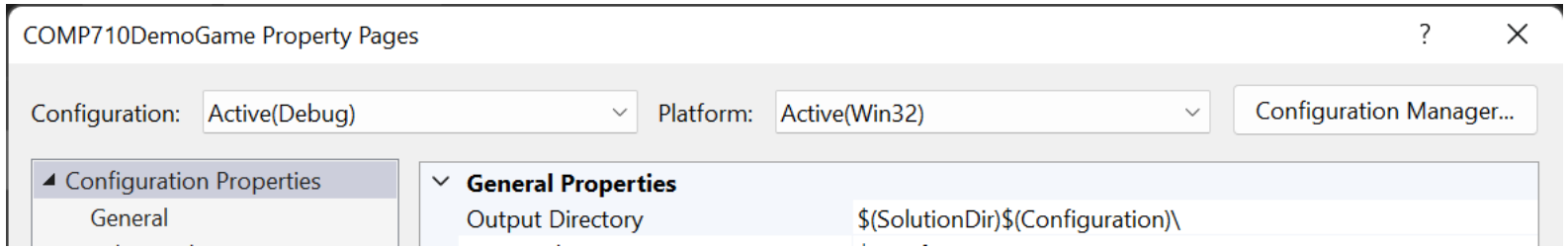
The first setting to change…

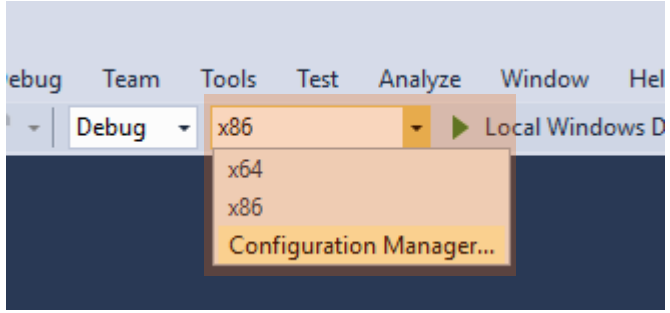
Delete the “x64” target! ☺

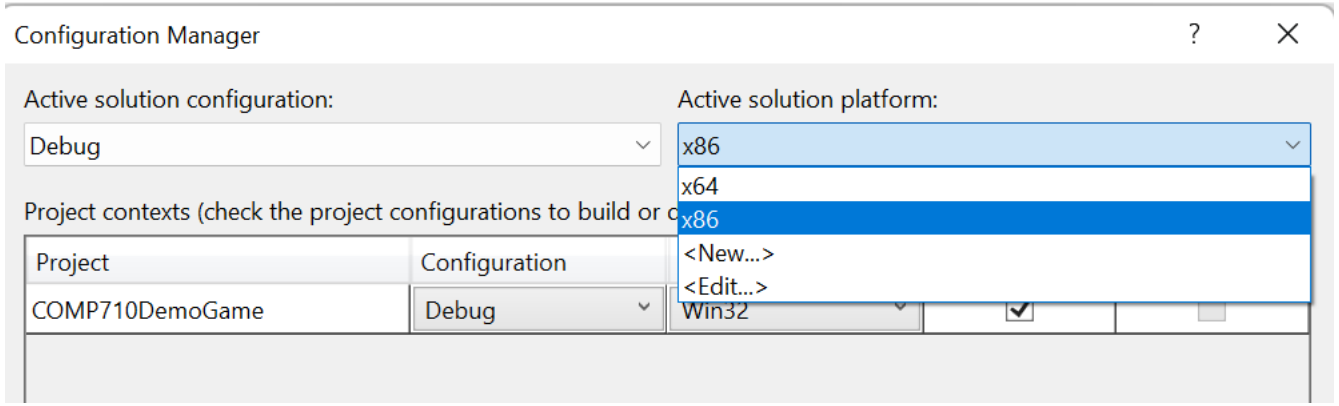
We will only use 32-bit…

Game Production: Project Configuration

- First go to the: “Configuration Manager….”



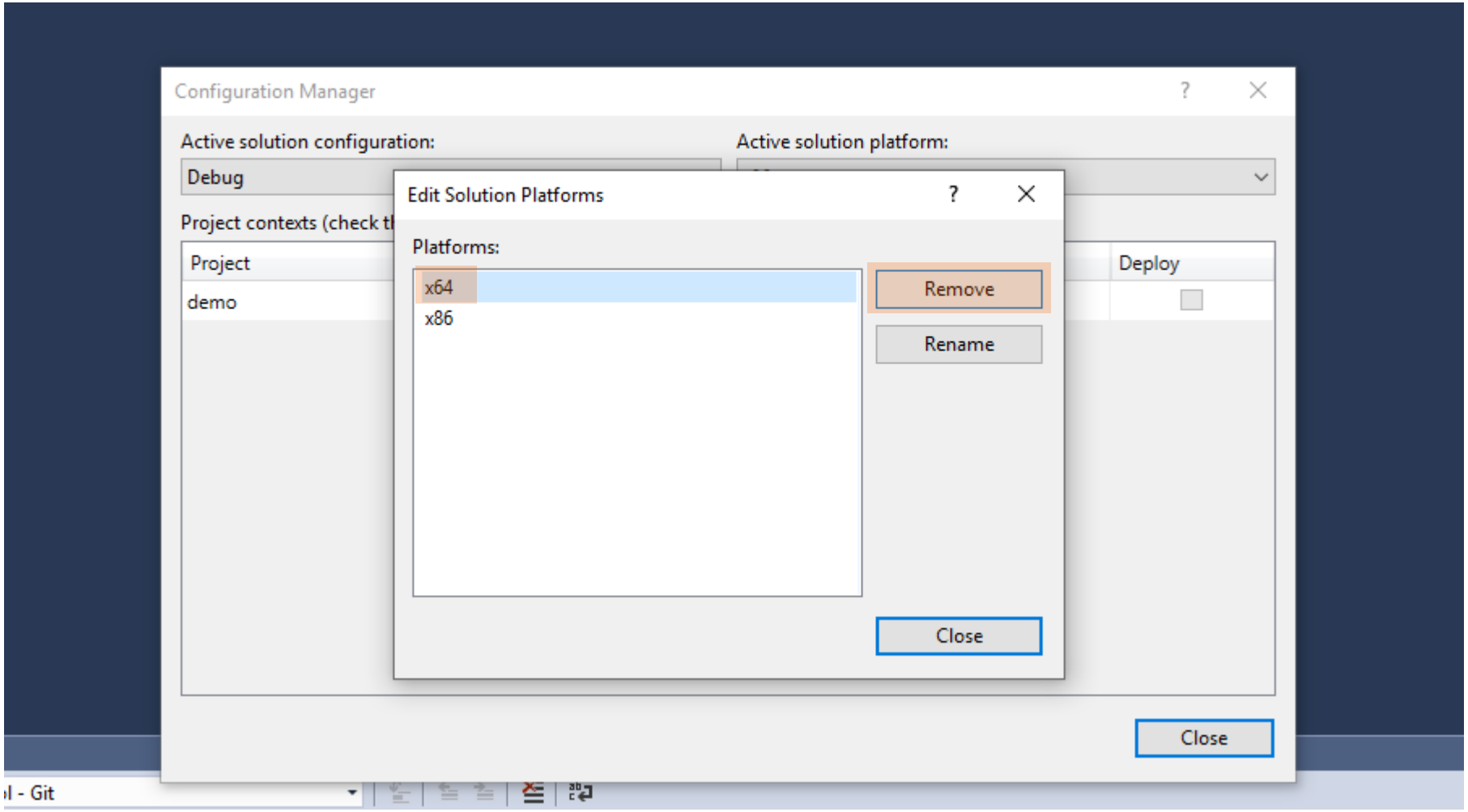




- Then choose “<Edit…>” on the “Active solution platform:” option…

Game Production: Project Configuration

- Finally…
- Choose: x64
- And then click: “Remove”
- ☺



Next, we will configure paths and naming…

In an effort to create a robust game program structure… ☺

Visual Studio…
“Output Directory”

Where the resulting `.exe`
is written when building…

Visual Studio…
“Intermediate Directory”

Where the `.obj` files are written when compiling…

Visual Studio…
“Target Name”

The name of the resulting `.exe` file…

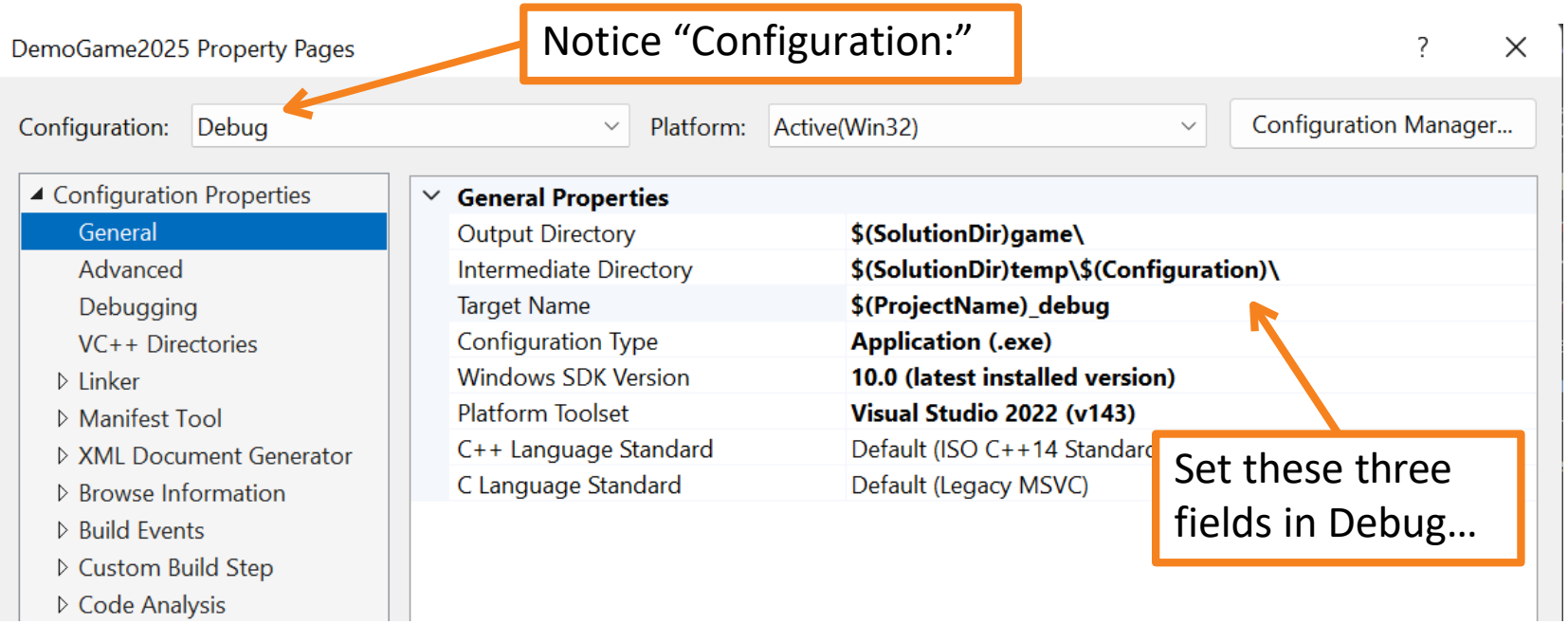
“Target Name”
Clearly name different build target .exe files…

Debug: **`codename_debug.exe`**

Release: **`codename.exe`**

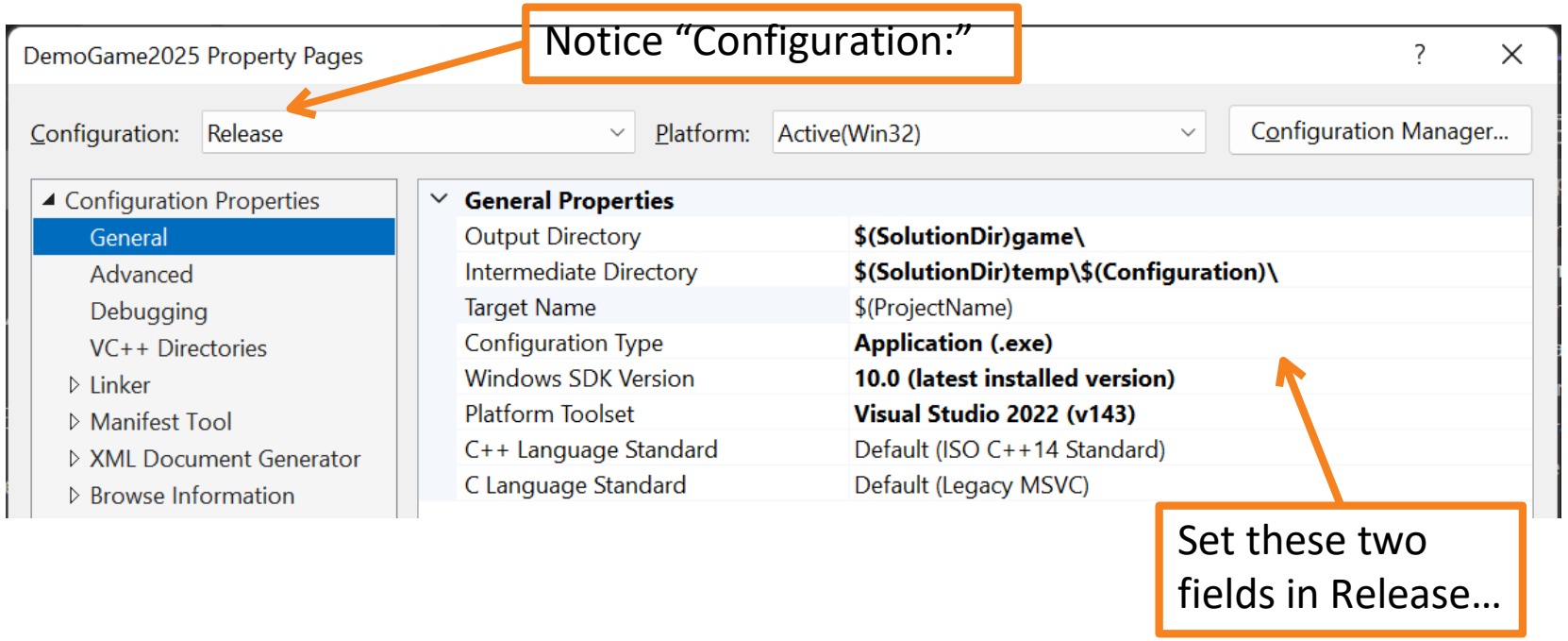
Debug Build Target Settings: General

- Output Directory:
  - Where does the `.exe` go?
- Intermediate Directory:
  - Where do the `.obj` files go?
- Target Name:
  - `…_debug.exe`



Release Build Target Settings: General

- Output Directory:
  - Where does the `.exe` go?
- Intermediate Directory:
  - Where do the `.obj` files go?
- Target Name:
  - … `.exe`



Visual Studio… “Working Directory”

Where the game will launch from… where its required assets will be located…

Visual Studio…
“Working Directory”

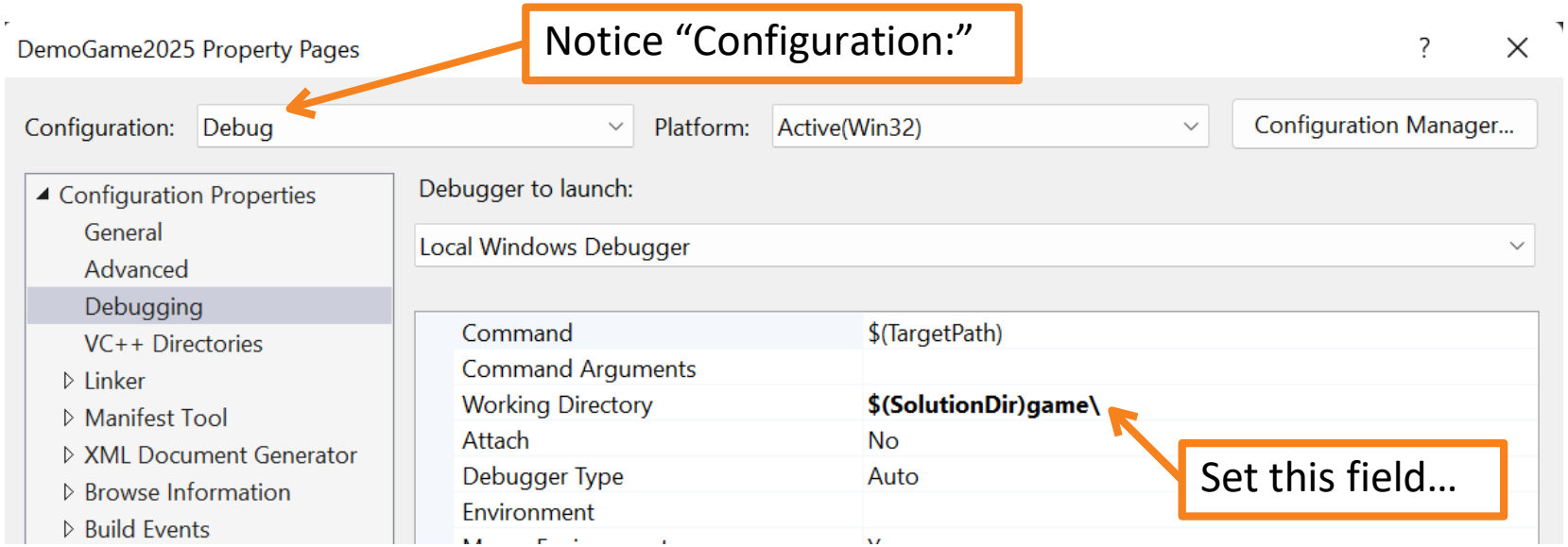
In the “game” folder, we will have all the required assets…

Visual Studio…
“Working Directory”

Both the Debug and Release executables will be written here…

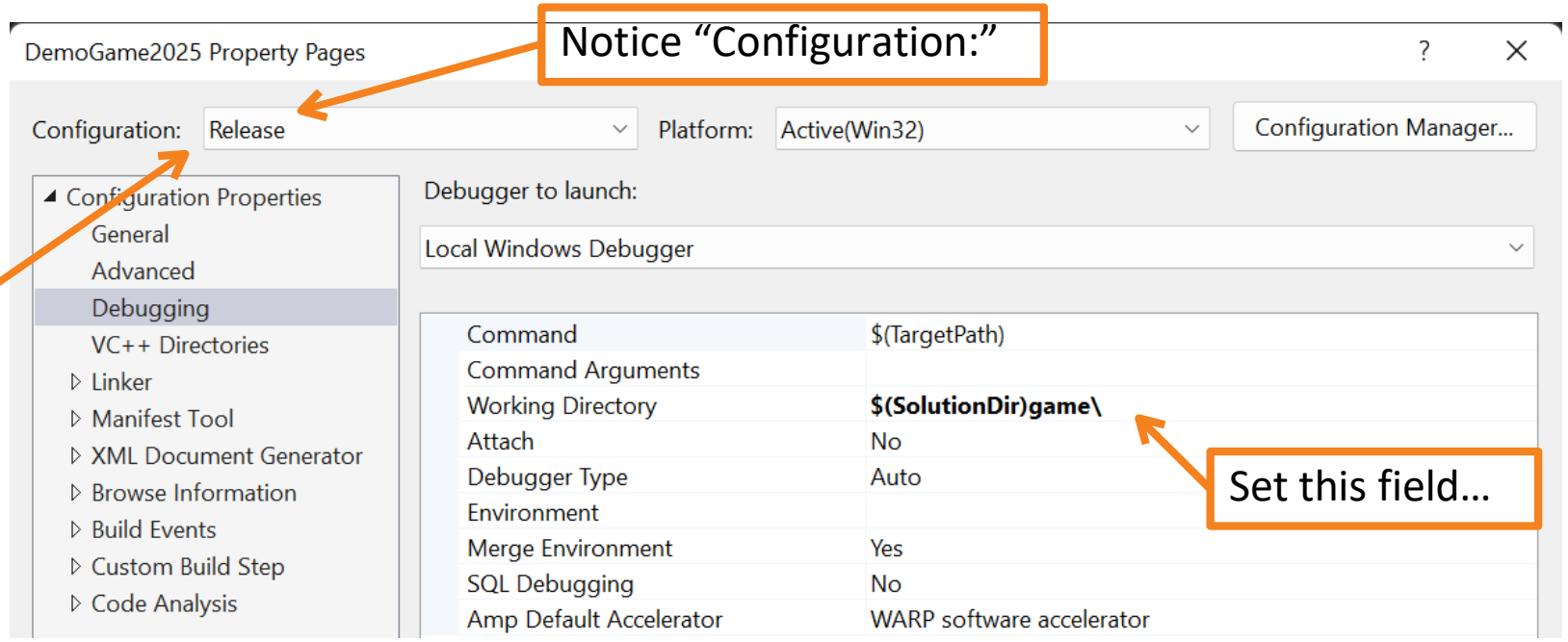
Debug Build Target Settings: Debugging

- Working Directory



Release Build Target Settings: Debugging

- Working Directory



And set this for the Release build target too…

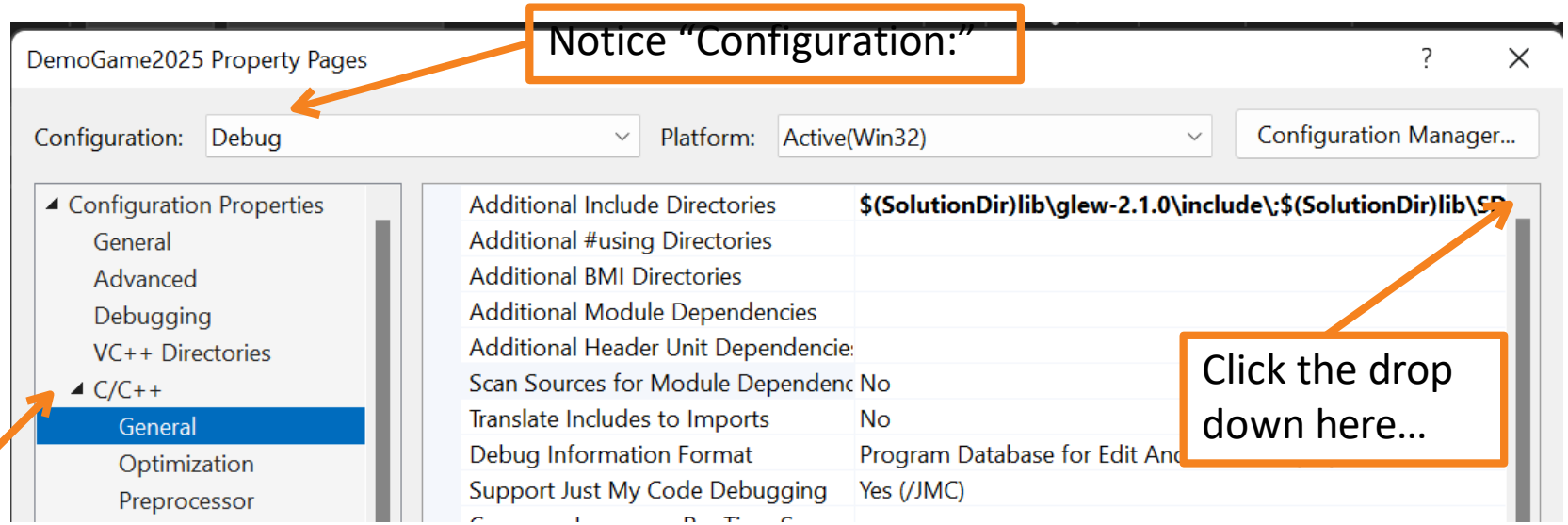
Additional includes…

Keep all paths relative…

No absolute paths!

Debug Build Target Settings: C/C++ General

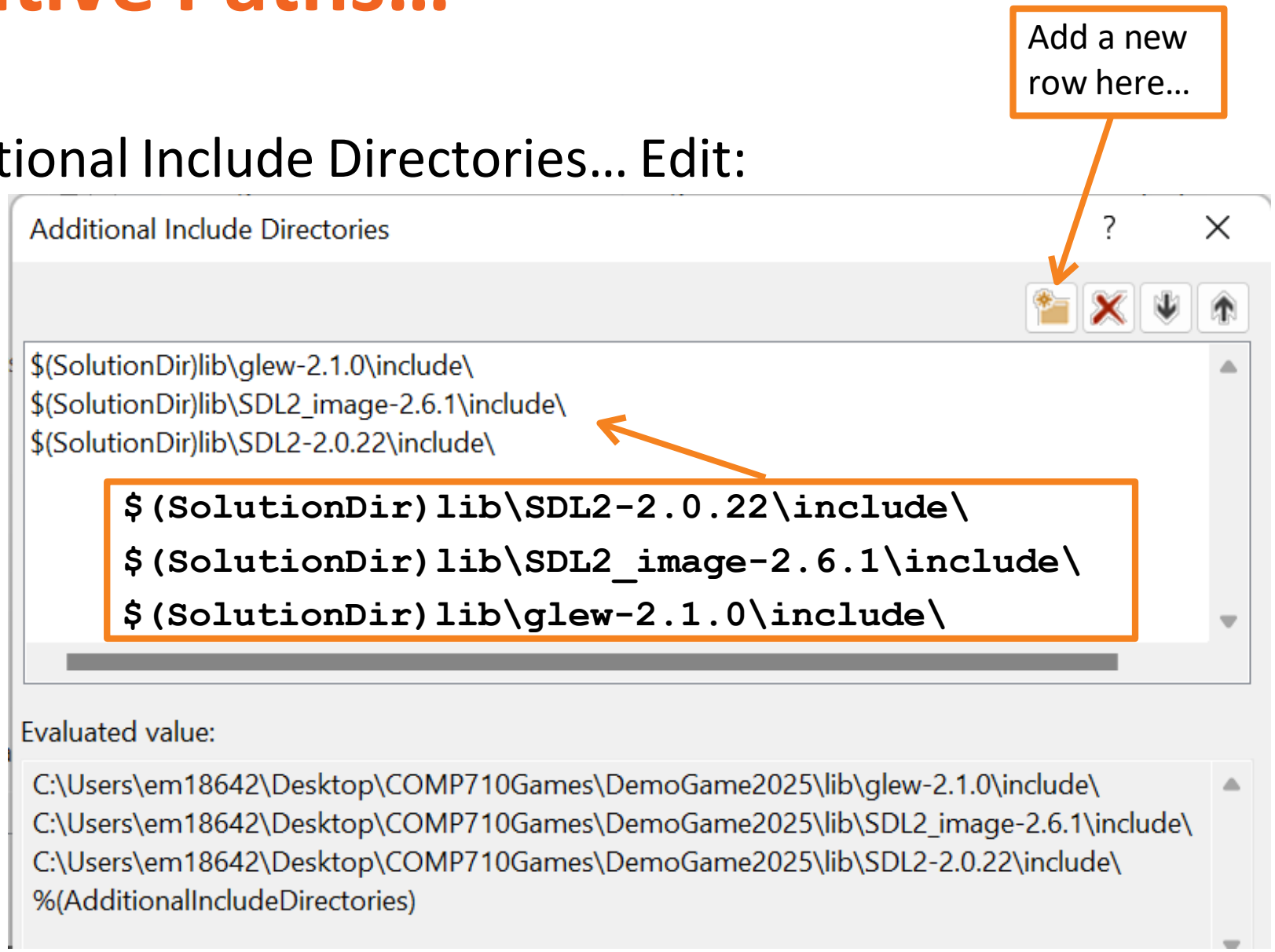
- Additional Include Directories



Note, to see the "C/C++" section, a `.cpp` file must have been add to the project! ☺

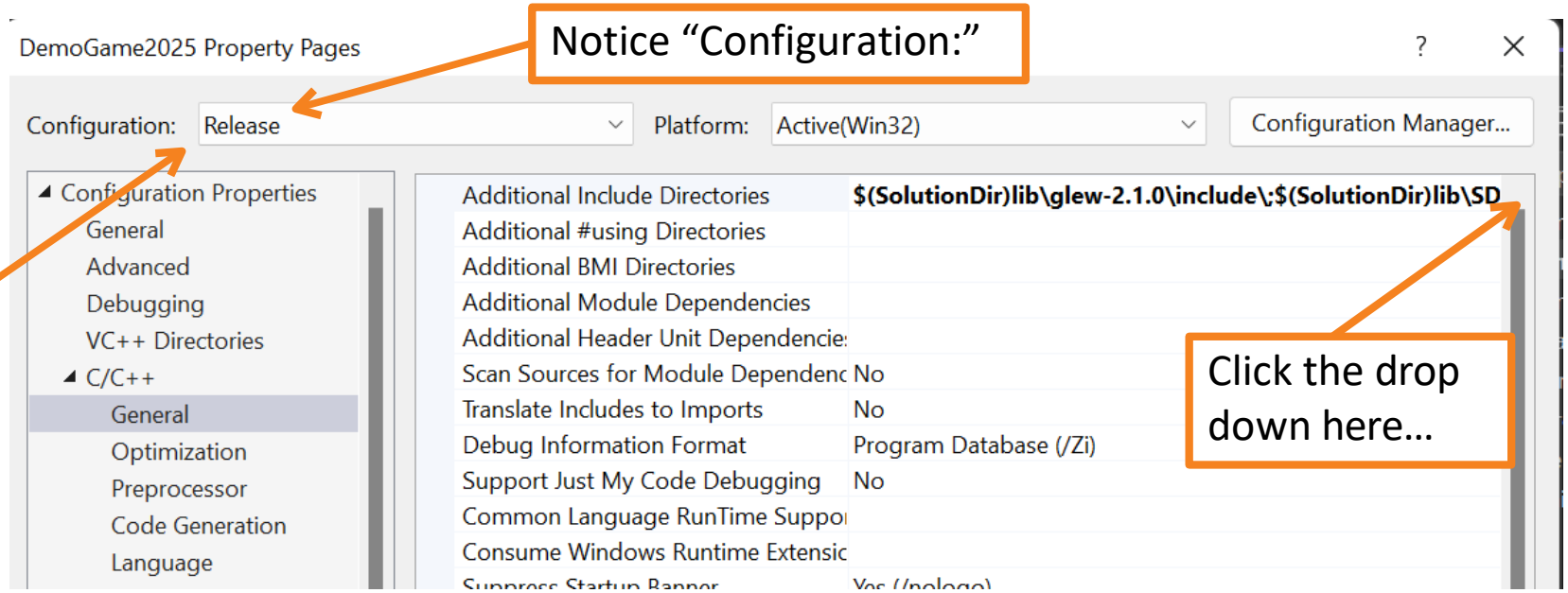
Relative Paths…

- Additional Include Directories… Edit:



Release Build Target Settings: C/C++ General

- Additional Include Directories



And set this for the Release build target too...

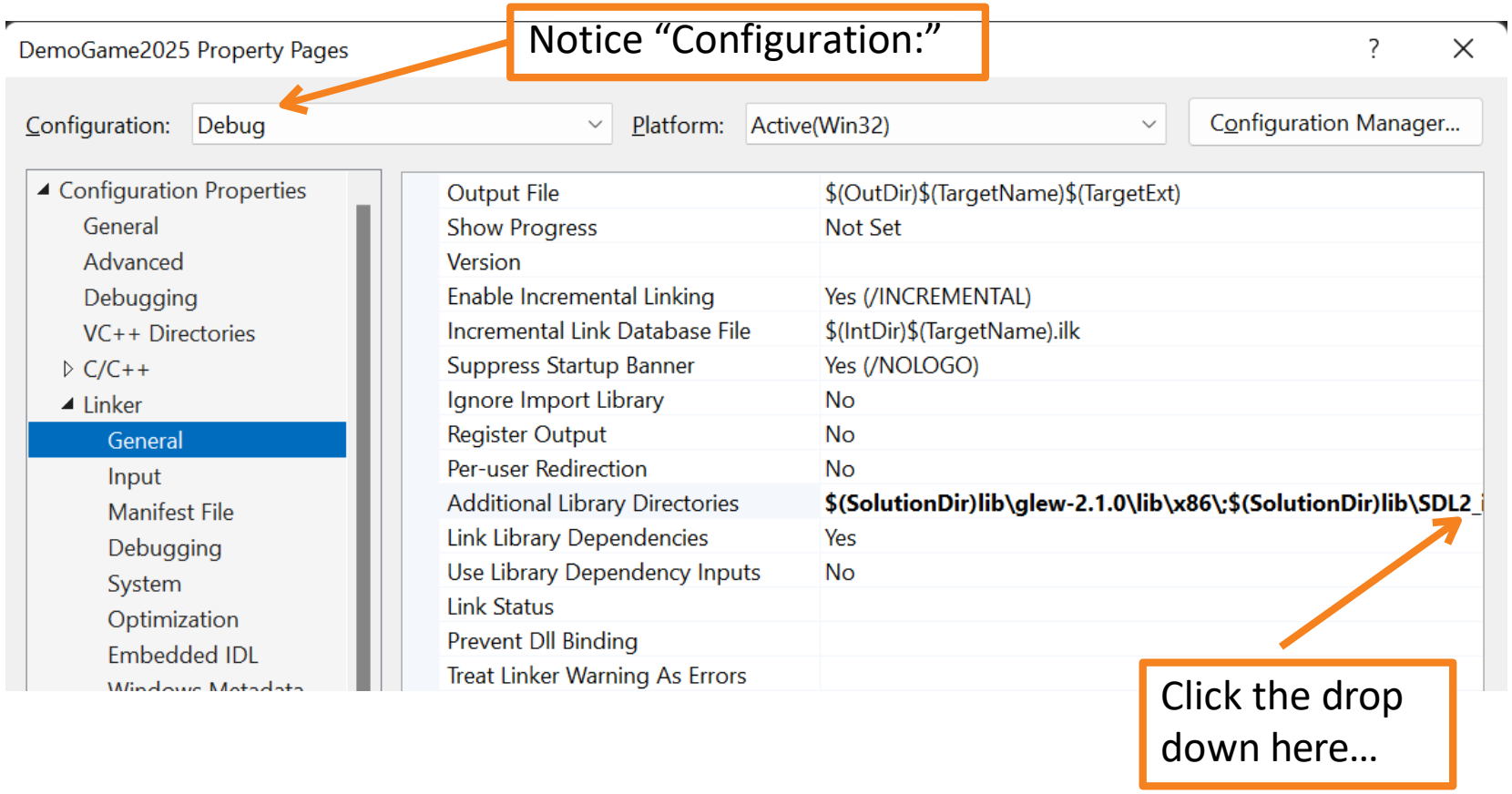
Additional libraries…

Keep all paths relative…

No absolute paths!

Debug Build Target Settings: Linker General

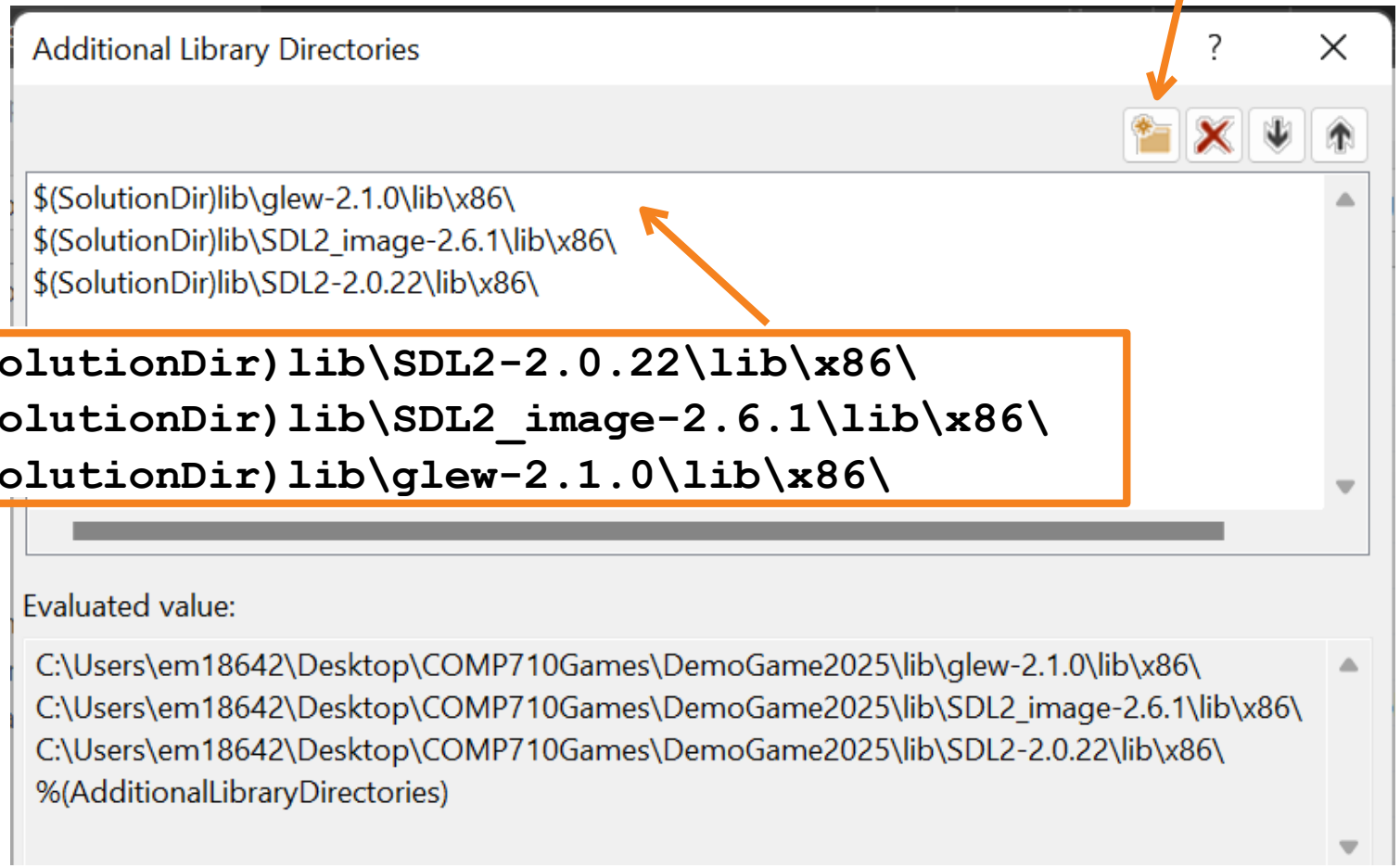
- Additional Library Directories



Relative Paths…

- Additional Library Directories… Edit:

Add a new row here…




```
$(SolutionDir)lib\SDL2-2.0.22\lib\x86\
$(SolutionDir)lib\SDL2_image-2.6.1\lib\x86\
$(SolutionDir)lib\glew-2.1.0\lib\x86\
```

Release Build Target Settings: Linker General

- Additional Library Directories

And set this for the Release build target too…

Notice "Configuration:"

Framework Property Pages

Configuration: Release Platform: Win32 Configuration Manager...

- Configuration Properties
  - General
  - Debugging
  - VC++ Directories
  - C/C++
  - Linker
    - General
    - Input
    - Manifest File
    - Debugging
    - System
    - Optimization
    - Embedded IDL
    - Windows Metadata
    - Advanced
    - All Options
    - Command Line
  - Manifest Tool
  - XML Document Generator
  - Browse Information
  - Build Events
  - Custom Build Step
  - Code Analysis

Output File	$(OutDir)$(TargetName)$(TargetExt)
Show Progress	Not Set
Version	
Enable Incremental Linking	**No (/INCREMENTAL:NO)**
Suppress Startup Banner	Yes (/NOLOGO)
Ignore Import Library	No
Register Output	No
Per-user Redirection	No
Additional Library Directories	**$(SolutionDir)..\..\..\shared\API-SDL\SDL2-devel-2.0.12-VC**
Link Library Dependencies	Yes
Use Library Dependency Inputs	No
Link Status	
Prevent Dll Binding	
Treat Linker Warning As Errors	
Force File Output	
Create Hot Patchable Image	
Specify Section Attributes	

Output File
The /OUT option overrides the default name and location of the program that the linker creates.

Click the drop down here…

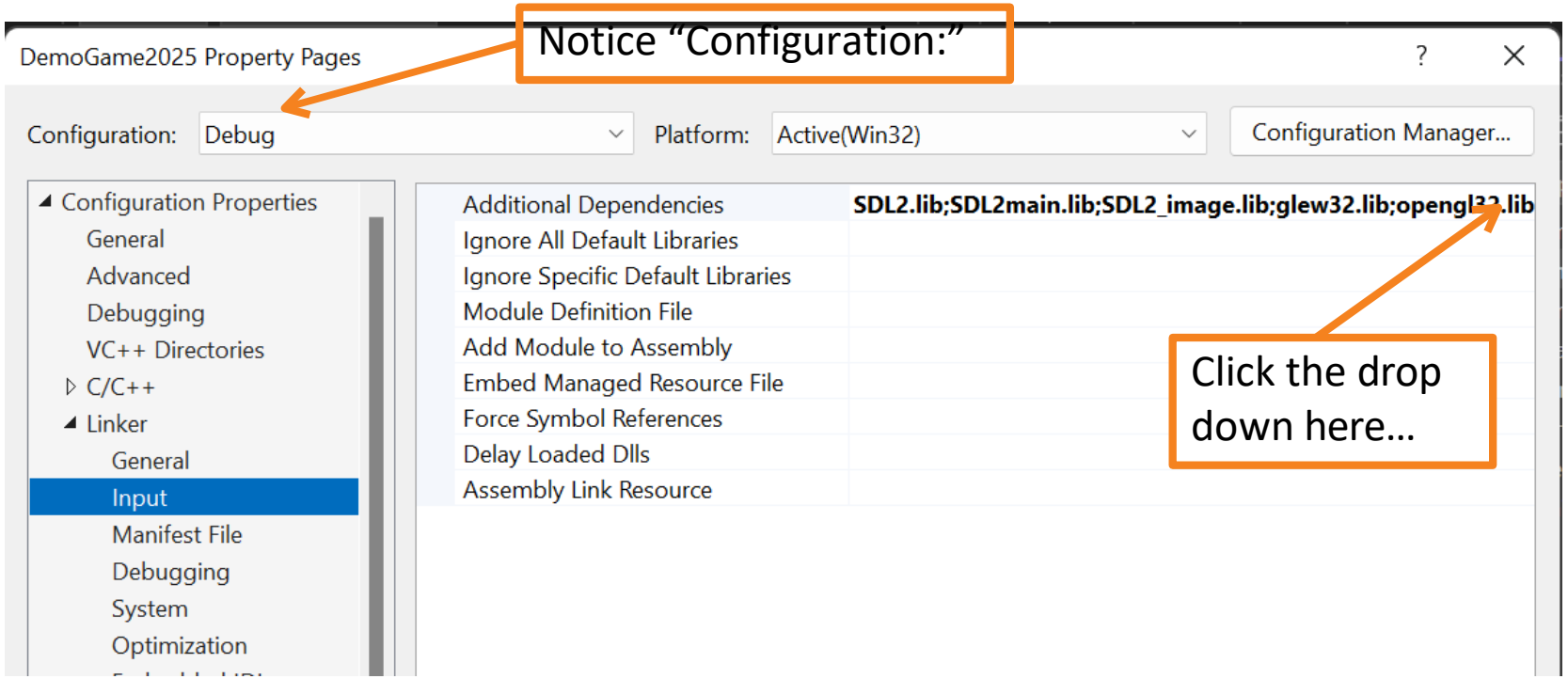
OK Cancel Apply

Additional libraries…

Don’t forget to add the required `.lib` files… ☺

Debug Build Target Settings: Input

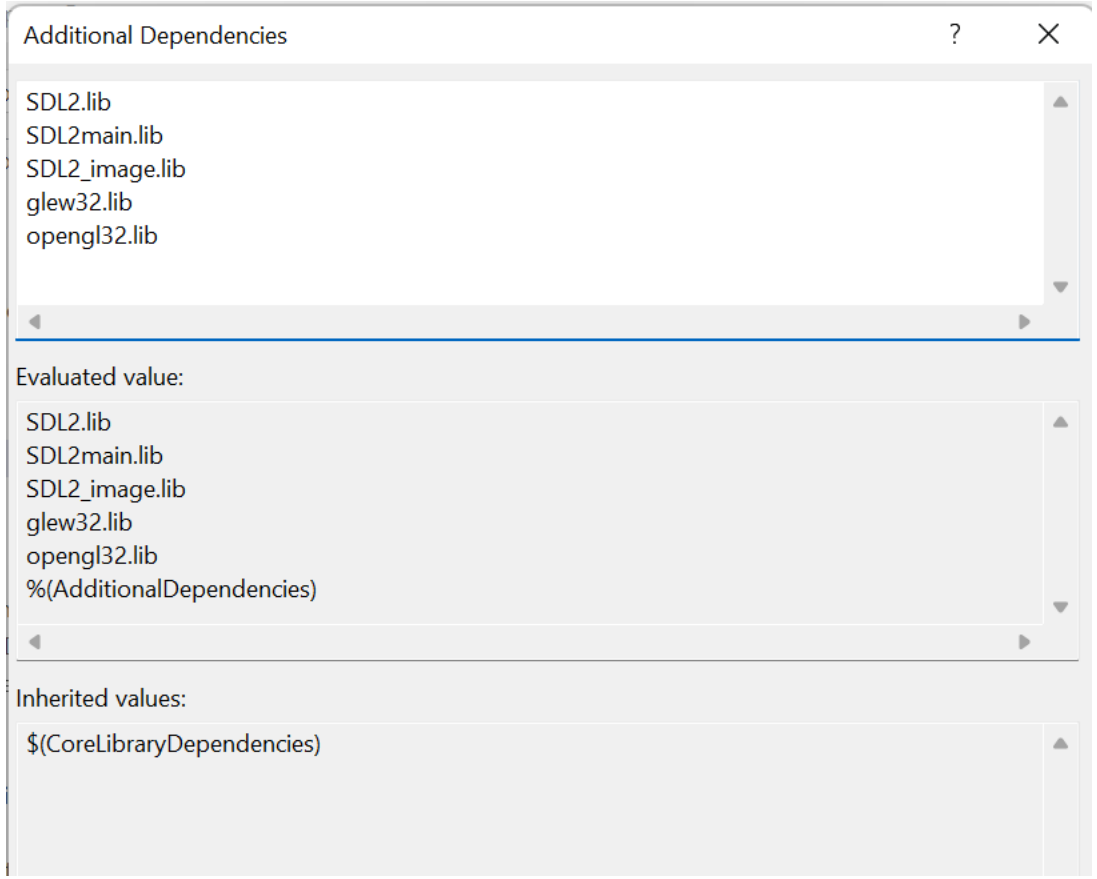
- Additional Dependencies



Additional Dependencies… the `.lib` files…

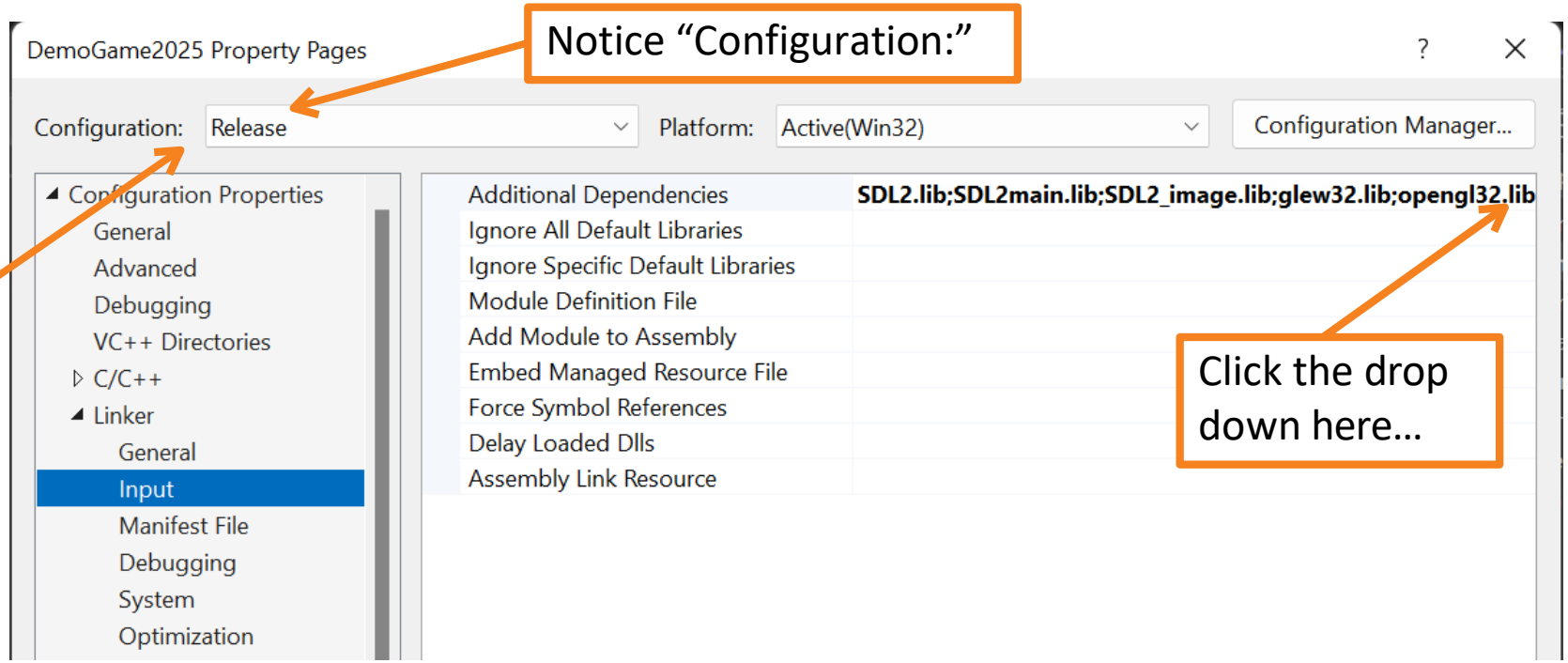
- Edit: Additional Dependencies…

```
SDL2.lib
SDL2main.lib
SDL2_image.lib
glew32.lib
opengl32.lib
```



Release Build Target Settings: Input

- Additional Dependencies



Ok… with project configuration done…

Next we can start building the reusable framework… ☺

Audit trails…

Logging…

Let’s start simple… ☺

The `LogManager` Class

- What does the **`LogManager`** do?
  - Report simple text-based error messages…
- Simple text audit trail…
  - Log to Visual Studio's Output window tab – no more console window! ☺
  - Logs simple C-String **`char*`** messages, by calling the **`Log()`** method.
  - We can construct messages using **`sprintf`** from the **`<cstdio>`** header…
    - i.e.: We can convert an **`int`** to text via **`sprintf`**'s **`%d`** (and a **`float`** with **`%f`** ☺)
- Design Pattern: Singleton
  - Only ever want one instance of this object!
    - And we can then log from anywhere in the project…

The LogManager Class (logmanager.h)

```
// COMP710 GP Framework 2025
#ifndef __LOGMANAGER_H
#define __LOGMANAGER_H

class LogManager
{
    // Member methods:
public:
    static LogManager& GetInstance();
    static void DestroyInstance();

    void Log(const char* pcMessage);
protected:

private:
    LogManager();
    ~LogManager();
    LogManager(const LogManager& logManager);
    LogManager& operator=(const LogManager& logManager);

    // Member data:
public:

protected:
    static LogManager* sm_pInstance;

private:

};
```

Design Pattern: Singleton

The static **`GetInstance`** method…

Design Pattern: Singleton

Private constructor… ☺

COMP710 Coding Standards:

Prefix static member fields with `sm_`.

Design Pattern: Singleton

The static instance of this class – only one **`LogManager`** object can ever be created!

The logmanager.cpp Implementation…

```
// COMP710 GP Framework

// This include:
#include "logmanager.h"

// Library includes:
#include <Windows.h>

// Static Members:
LogManager* LogManager::sm_pInstance = 0;

LogManager& LogManager::GetInstance()
{
    if (sm_pInstance == 0)
    {
        sm_pInstance = new LogManager();
    }


    return (*sm_pInstance);
}

void
LogManager::DestroyInstance()
{
    delete sm_pInstance;
    sm_pInstance = 0;
}
```

Design Pattern: Singleton

Set static singleton instance pointer to null at the start of the program…

Design Pattern: Singleton

If the instance does not yet exist… create it!

Design Pattern: Singleton

Return the singleton object instance…

Design Pattern: Singleton

Deallocate the instance to clean up the memory allocation at the end of the game…

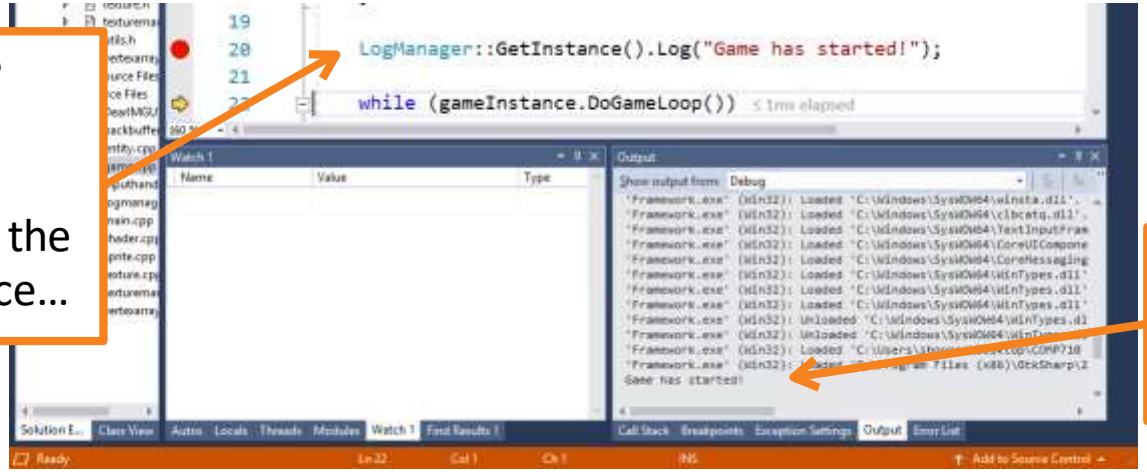
The `logmanager.cpp` Implementation…

```
LogManager::LogManager()
{

}

LogManager::~LogManager()
{

}

void LogManager::Log(const char* pcMessage)
{
    OutputDebugStringA(pcMessage);
    OutputDebugStringA("\n");
}
```

The **`Log`** method, takes in a C-String message, and sends it to Visual Studio's Output window tab…

An example call in the **`main`** function…

Sending a message to the **`LogManager`** instance…



See the message output by the **`LogManager`** in the Output tab…

The Game Object…

The object which controls the game loop… (the processing and drawing of game objects…) ☺

The Game Class (game . h)

```
// COMP710 GP Framework
#ifndef __GAME_H
#define __GAME_H

// Forward declarations:
class Renderer;

class Game
{
    // Member methods:
public:
    static Game& GetInstance();
    static void DestroyInstance();

    bool Initialise();
    bool DoGameLoop();
    void Quit();

protected:
    void Process(float deltaTime);
    void Draw(Renderer& renderer);

    void ProcessFrameCounting(float deltaTime);

private:
    Game();
    ~Game();
    Game(const Game& game);
    Game& operator=(const Game& game);
```

Can add a **#define USE_LAG** to activate lag loop…

Design Pattern: Singleton

The static **GetInstance** method…

The initialisation, game loop and quit methods…

The **Process** and **Draw** methods…

Stop accidental object copies… ☺

The Game Class (game.h)

```
    // Member data:
public:

protected:
    static Game* sm_pInstance;
    Renderer* m_pRenderer;

    __int64 m_iLastTime;
    float m_fExecutionTime;
    float m_fElapsedSeconds;
    int m_iFrameCount;
    int m_iFPS;

#ifdef USE_LAG
    float m_fLag;
    int m_iUpdateCount;
#endif // USE_LAG

    bool m_bLooping;

private:

};

#endif // __GAME_H
```

The Renderer…

Some frame counting data…

Is the game loop running? ☺

The game.cpp Implementation…

```
// COMP710 GP Framework 2022

// This include:
#include "game.h"

// Library includes:
#include "renderer.h"
#include "logmanager.h"

// Static Members:
Game* Game::sm_pInstance = 0;

Game& Game::GetInstance()
{
    if (sm_pInstance == 0)
    {

        sm_pInstance = new Game();
    }

    return (*sm_pInstance);
}

void Game::DestroyInstance()
{
    delete sm_pInstance;
    sm_pInstance = 0;
}
```

Design Pattern: Singleton ☺

The game.cpp Implementation…

```
Game::Game(): m_pRenderer(0), m_bLooping(true)
{

}

Game::~Game()
{
    delete m_pRenderer;
    m_pRenderer = 0;
}

void Game::Quit()
{
    m_bLooping = false;
}
```

Clean up the renderer allocation…

The game.cpp Implementation…

```
bool Game::Initialise()
{
    int bbWidth = 1024;
    int bbHeight = 768;

    m_pRenderer = new Renderer();
    if (!m_pRenderer->Initialise(true, bbWidth, bbHeight))
    {
        LogManager::GetInstance().Log("Renderer failed to initialise!");
        return false;
    }


    bbWidth = m_pRenderer->GetWidth();
    bbHeight = m_pRenderer->GetHeight();

    m_iLastTime = SDL_GetPerformanceCounter();

    m_pRenderer->SetClearColour(0, 255, 255);

    return true;
}
```

Desired backbuffer dimensions… e.g. resolution of 1024 by 768…

Initialise in windowed mode…

Get the backbuffer size that was actual created…

Set up the initial time value…

Set the backbuffer clear colour…

The game.cpp Implementation…

```
bool Game::DoGameLoop()
{
    const float stepSize = 1.0f / 60.0f;


    // TODO: Process input here!
    SDL_Event event;
    while (SDL_PollEvent(&event) != 0)
    {
        continue;
    }

    if (m_bLooping)
    {
        Uint64 current = SDL_GetPerformanceCounter();
        float deltaTime = (current - m_iLastTime) / static_cast<float>(SDL_GetPerformanceFrequency());

        m_iLastTime = current;

        m_fExecutionTime += deltaTime;

        Process(deltaTime);

#ifdef USE_LAG
        m_fLag += deltaTime;

        int innerLag = 0;
```

Temporary stub to process SDL events… we will modify this in the next studio session…

The game loop cycle! ☺

Update the timing…

Process the game world…

The game.cpp Implementation…

```
        while (m_fLag >= stepSize)
        {
            Process(stepSize);

            m_fLag -= stepSize;

            ++m_iUpdateCount;
            ++innerLag;
        }
#endif //USE_LAG

        Draw(*m_pRenderer);
    }

    return m_bLooping;
}

void Game::Process(float deltaTime)
{
    ProcessFrameCounting(deltaTime);


    // TODO: Add game objects to process here!
}
```

Draw the game world…

Update frame counting data…

Processing of game objects will occur here…

The game.cpp Implementation…

```
void Game::Draw(Renderer& renderer)
{

    ++m_iFrameCount;

    renderer.Clear();

    // TODO: Add game objects to draw here!

    renderer.Present();
}

void
Game::ProcessFrameCounting(float deltaTime)
{
    // Count total simulation time elapsed:
    m_fElapsedSeconds += deltaTime;

    // Frame Counter:
    if (m_fElapsedSeconds > 1.0f)
    {
        m_fElapsedSeconds -= 1.0f;
        m_iFPS = m_iFrameCount;
        m_iFrameCount = 0;
    }
}
```

Clear the backbuffer…

Drawing of game objects will occur here…

Promote the backbuffer to be the frontbuffer… ☺

Compute frames per second…

The `main` function…

The code that actually starts the game framework running… ☺

The `main` Function in `main.cpp`

```
// COMP710 GP Framework 2022

// Library includes:
#include <SDL.h>

// Local includes:
#include "game.h"
#include "logmanager.h"

int main(int argc, char* argv[])
{
    Game& gameInstance = Game::GetInstance();
    if (!gameInstance.Initialise())
    {
        LogManager::GetInstance().Log("Game initialise failed!");

        return 1;
    }

    while (gameInstance.DoGameLoop())
    {
        // No body.
    }

    Game::DestroyInstance();
    LogManager::DestroyInstance();

    return 0;
}
```

Must **`#include <SDL.h>`** to bind in the SDL **`main`** start-up code…

Since **`main.cpp`** uses the **`Game`** interface and the **`LogManager`** interface, we must **`#include`** these header files…

Create the **`Game`** object instance…

Initialise the game…

Log the failure… ☹

This is where I added the previous example of calling **`Log`** to note the game had started successfully… ☺

Run the game loop…

Destroy the singleton object instances before quitting…

The `Renderer` class…

The object that can draw to the screen… ☺

The Renderer Class

```
// COMP710 GP Framework 2022
#ifndef __RENDERER_H
#define __RENDERER_H

// Forward Declarations:
class TextureManager;
class Shader;
class VertexArray;
class Sprite;
struct SDL_Window;

// Library includes:
#include <SDL.h>

class Renderer
{
    // Member methods:
public:
    Renderer();
    ~Renderer();

    bool Initialise(bool windowed, int width = 0, int height = 0);

    void Clear();
    void Present();

    void SetClearColour(unsigned char r, unsigned char g, unsigned char b);
    void GetClearColour(unsigned char& r, unsigned char& g, unsigned char& b);
```

Types used (pointers or references) by this class declaration…

Public interface….

The Renderer Class

```
    int GetWidth() const;
    int GetHeight() const;

    Sprite* CreateSprite(const char* pcFilename);
    void DrawSprite(Sprite& sprite);

protected:
    bool InitialiseOpenGL(int screenWidth, int screenHeight);
    void SetFullscreen(bool fullscreen);

    void LogSdlError();

    bool SetupSpriteShader();

private:
    Renderer(const Renderer& renderer);
    Renderer& operator=(const Renderer& renderer);

    // Member data:
public:

protected:
    TextureManager* m_pTextureManager;
    SDL_Window* m_pWindow;
    SDL_GLContext m_glContext;

    Shader* m_pSpriteShader;
    VertexArray* m_pSpriteVertexData;
```

Public interface….

Encapsulated interface….

Stop accidental object copies… ☺

The Renderer Class

```
    int m_iWidth;
    int m_iHeight;

    float m_fClearRed;
    float m_fClearGreen;
    float m_fClearBlue;

private:

};

#endif // __RENDERER_H
```

The renderer.cpp Implementation…

```
// COMP710 GP Framework 2022

// This include:
#include "renderer.h"

// Local includes:
#include "texturemanager.h"
#include "logmanager.h"
#include "shader.h"
#include "vertexarray.h"
#include "sprite.h"
#include "matrix4.h"

// Library includes:
#include <SDL.h>
#include <SDL_image.h>
#include <glew.h>
#include <cassert>
#include <cmath>
```

COMP710 Coding Standards:

When including header files…

Start with the local header (i.e. `renderer.cpp` includes `renderer.h`) first.

Then include the required headers that are local to the project… (i.e. header files enclosed in " " quotes).

Then include the library headers… (i.e. headers enclosed in < > brackets).

Remember the C++ Preprocessor will process the `.cpp` file from top to bottom… each `#include` is adding more declarations and prototypes… ☺

The renderer.cpp Implementation…

```
Renderer::Renderer()
: m_pTextureManager(0)
, m_pSpriteShader(0)
, m_pSpriteVertexData(0)
, m_glContext(0)
, m_iWidth(0)
, m_iHeight(0)
, m_fClearRed(0.0f)
, m_fClearGreen(0.0f)
, m_fClearBlue(0.0f)
{

}

Renderer::~Renderer()
{
    delete m_pSpriteShader;
    m_pSpriteShader = 0;

    delete m_pSpriteVertexData;
    m_pSpriteVertexData = 0;

    delete m_pTextureManager;
    m_pTextureManager = 0;

    SDL_DestroyWindow(m_pWindow);
    IMG_Quit();
    SDL_Quit();
}
```

The renderer.cpp Implementation…

```
bool Renderer::Initialise(bool windowed, int width, int height)
{
    if (SDL_Init(SDL_INIT_VIDEO) < 0)
    {
        LogSdlError();
        return false;
    }

    if (!windowed)
    {
        // Go fullscreen, with current resolution!
        int numDisplays = SDL_GetNumVideoDisplays();
        SDL_DisplayMode* currentDisplayMode = new SDL_DisplayMode[numDisplays];

        for (int k = 0; k < numDisplays; ++k)
        {
            int result = SDL_GetCurrentDisplayMode(k, &currentDisplayMode[k]);
        }

        // Use the widest display?
        int widest = 0;
        int andItsHeight = 0;

        for (int k = 0; k < numDisplays; ++k)
        {
            if (currentDisplayMode[k].w > widest)
            {
                widest = currentDisplayMode[k].w;
```

The renderer.cpp Implementation…

```
                andItsHeight = currentDisplayMode[k].h;
            }
        }

        delete[] currentDisplayMode;
        currentDisplayMode = 0;

        width = widest;
        height = andItsHeight;
    }

    bool initialised = InitialiseOpenGL(width, height);

    SetFullscreen(!windowed);

    if (initialised)
    {
        m_pTextureManager = new TextureManager();
        assert(m_pTextureManager);
        initialised = m_pTextureManager->Initialise();
    }

    return initialised;
}
```

The renderer.cpp Implementation…

```
bool Renderer::InitialiseOpenGL(int screenWidth, int screenHeight)
{
    m_iWidth = screenWidth;
    m_iHeight = screenHeight;

    m_pWindow = SDL_CreateWindow("COMP710 GP Framework 2025", SDL_WINDOWPOS_UNDEFINED,
                                 SDL_WINDOWPOS_UNDEFINED, screenWidth, screenHeight, SDL_WINDOW_OPENGL);

    SDL_GL_SetAttribute(SDL_GL_CONTEXT_PROFILE_MASK, SDL_GL_CONTEXT_PROFILE_CORE);
    SDL_GL_SetAttribute(SDL_GL_CONTEXT_MAJOR_VERSION, 3);
    SDL_GL_SetAttribute(SDL_GL_CONTEXT_MINOR_VERSION, 3);

    SDL_GL_SetAttribute(SDL_GL_RED_SIZE, 8);
    SDL_GL_SetAttribute(SDL_GL_GREEN_SIZE, 8);
    SDL_GL_SetAttribute(SDL_GL_BLUE_SIZE, 8);
    SDL_GL_SetAttribute(SDL_GL_ALPHA_SIZE, 8);

    SDL_GL_SetAttribute(SDL_GL_DOUBLEBUFFER, 1);
    SDL_GL_SetAttribute(SDL_GL_ACCELERATED_VISUAL, 1);

    m_glContext = SDL_GL_CreateContext(m_pWindow);

    GLenum glewResult = glewInit();
```

The renderer.cpp Implementation…

```
    if (glewResult != GLEW_OK)
    {
        return false;
    }

    // Disable VSYNC
    SDL_GL_SetSwapInterval(0);

    bool shadersLoaded = SetupSpriteShader();

    return shadersLoaded;
}
```

The renderer.cpp Implementation…

```
void Renderer::Clear()
{
    glClearColor(m_fClearRed, m_fClearGreen, m_fClearBlue, 1.0f);
    glClear(GL_COLOR_BUFFER_BIT);
}


void Renderer::Present()
{
    SDL_GL_SwapWindow(m_pWindow);
}


void Renderer::SetFullscreen(bool fullscreen)
{
    if (fullscreen)
    {

        // SDL_SetWindowFullscreen(m_pWindow, SDL_WINDOW_FULLSCREEN | SDL_WINDOW_ALWAYS_ON_TOP);
        SDL_SetWindowFullscreen(m_pWindow, SDL_WINDOW_FULLSCREEN_DESKTOP | SDL_WINDOW_ALWAYS_ON_TOP);
        SDL_SetHint(SDL_HINT_VIDEO_MINIMIZE_ON_FOCUS_LOSS, "0");
        SDL_SetWindowSize(m_pWindow, m_iWidth, m_iHeight);
    }
    else
    {
        SDL_SetWindowFullscreen(m_pWindow, 0);
    }
}
```

Clear the backbuffer…

Promote the backbuffer to become the frontbuffer…

Set up fullscreen or windowed mode…

We may need to tweak this further… ☺

The renderer.cpp Implementation…

```
void Renderer::SetClearColour(unsigned char r, unsigned char g, unsigned char b)
{

    m_fClearRed = r / 255.0f;
    m_fClearGreen = g / 255.0f;
    m_fClearBlue = b / 255.0f;
}

void Renderer::GetClearColour(unsigned char& r, unsigned char& g, unsigned char& b)
{

    r = static_cast<unsigned char>(m_fClearRed * 255.0f);
    g = static_cast<unsigned char>(m_fClearGreen * 255.0f);
    b = static_cast<unsigned char>(m_fClearBlue * 255.0f);
}

int Renderer::GetWidth() const
{
    return m_iWidth;
}


int Renderer::GetHeight() const
{
    return m_iHeight;
}
```

Set the clear colour…

Get the clear colour…

Other encapsulated accessor methods…

The renderer.cpp Implementation…

```
Sprite* Renderer::CreateSprite(const char* pcFilename)
{
    assert(m_pTextureManager);

    Texture* pTexture = m_pTextureManager->GetTexture(pcFilename);

    Sprite* pSprite = new Sprite();
    if (!pSprite->Initialise(*pTexture))
    {
       LogManager::GetInstance().Log("Sprite Failed to Create!");
    }

    return (pSprite);
}


void Renderer::LogSdlError()
{
    LogManager::GetInstance().Log(SDL_GetError());
}
```

Design Pattern: Object Factory

This method instantiates **`Sprite`** objects…

The caller must deallocate them with **`delete`** when they are no longer needed…

Helper method for logging SDL errors…

The renderer.cpp Implementation…

```
bool Renderer::SetupSpriteShader()
{
    m_pSpriteShader = new Shader();


    bool loaded = m_pSpriteShader->Load("shaders\\sprite.vert", "shaders\\sprite.frag");

    m_pSpriteShader->SetActive();

    float vertices[] =
    {
        -0.5f,  0.5f, 0.0f, 0.0f, 0.0f, // Top left
         0.5f,  0.5f, 0.0f, 1.0f, 0.0f, // Top right
         0.5f, -0.5f, 0.0f, 1.0f, 1.0f, // Bottom right
        -0.5f, -0.5f, 0.0f, 0.0f, 1.0f  // Bottom left
    };

    unsigned int indices[] = { 0, 1, 2, 2, 3, 0 };

    m_pSpriteVertexData = new VertexArray(vertices, 4, indices, 6);

    return loaded;
}
```

These two files must be in the **game\shaders** directory…

Loads the vertex and fragment shader code for rendering sprite with the OpenGL 3.3 programmable pipeline…

Sets up the vertex data used to rendering a sprite…

The renderer.cpp Implementation…

```
void Renderer::DrawSprite(Sprite& sprite)
{
    m_pSpriteShader->SetActive();
    m_pSpriteVertexData->SetActive();


    float angleInDegrees = sprite.GetAngle();

    float sizeX = static_cast<float>(sprite.GetWidth());
    float sizeY = static_cast<float>(sprite.GetHeight());

    const float PI = 3.14159f;
    float angleInRadians = (angleInDegrees * PI) / 180.0f;

    Matrix4 world;
    SetIdentity(world);
    world.m[0][0] = cosf(angleInRadians) * (sizeX);
    world.m[0][1] = -sinf(angleInRadians) * (sizeX);
    world.m[1][0] = sinf(angleInRadians) * (sizeY);
    world.m[1][1] = cosf(angleInRadians) * (sizeY);
    world.m[3][0] = static_cast<float>(sprite.GetX());
    world.m[3][1] = static_cast<float>(sprite.GetY());

    m_pSpriteShader->SetMatrixUniform("uWorldTransform", world);
```

Public interface method to draw a sprite to the backbuffer…

Activates the required shader programs on the graphics processing unit (GPU)

Gathers sprite data…

Creates a world transform matrix for the sprite… We will update this in the future too!

Sends the world matrix to the GPU….

The renderer.cpp Implementation…

```
    Matrix4 orthoViewProj;
    CreateOrthoProjection(orthoViewProj, static_cast<float>(m_iWidth), static_cast<float>(m_iHeight));

    m_pSpriteShader->SetVector4Uniform("colour", sprite.GetRedTint(),
                                                 sprite.GetGreenTint(),
                                                 sprite.GetBlueTint(),
                                                 sprite.GetAlpha());

    m_pSpriteShader->SetMatrixUniform("uViewProj", orthoViewProj);

    glEnable(GL_BLEND);
    glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA);

    glDrawElements(GL_TRIANGLES, 6, GL_UNSIGNED_INT, 0);
}
```

Create a basic 2D orthographic view/projection matrix…

Sends the colour blending values to the GPU

Sends the view/projection matrix to the GPU

Sets up the alpha blending mode

Draws the triangles with the GPU…

The `Texture` class…

The object that stores texture information…

The Texture Class (texture.h)

```
// COMP710 GP Framework 2025
#ifndef __TEXTURE_H
#define __TEXTURE_H

class Texture
{
    // Member methods:
public:
    Texture();
    ~Texture();

    bool Initialise(const char* pcFilename);

    void SetActive();

    int GetWidth() const;
    int GetHeight() const;

protected:

private:
    Texture(const Texture& texture);
    Texture& operator=(const Texture& texture);

    // Member data:
public:
```

Load a texture file…

Activate the texture on the GPU

Stop accidental object copies… ☺

The Texture Class (texture.h)

```
protected:
    unsigned int m_uiTextureId;
    int m_iWidth;
    int m_iHeight;

private:

};

#endif // __TEXTURE_H
```

The texture.cpp Implementation…

```
// COMP710 GP Framework 2025

// This include:
#include "texture.h"

// Local includes:
#include "logmanager.h"

// Library include:
#include <SDL_image.h>
#include <cassert>
#include "glew.h"

Texture::Texture()
: m_uiTextureId(0)
, m_iHeight(0)
, m_iWidth(0)
{

}

Texture::~Texture()
{
    glDeleteTextures(1, &m_uiTextureId);
}
```

The texture.cpp Implementation…

```
bool Texture::Initialise(const char* pcFilename)
{
    SDL_Surface* pSurface = IMG_Load(pcFilename);


    if (pSurface)
    {
        m_iWidth = pSurface->w;
        m_iHeight = pSurface->h;

        int bytesPerPixel = pSurface->format->BytesPerPixel;

        unsigned int format = 0;

        if (bytesPerPixel == 3)
        {
            format = GL_RGB;
        }
        else if (bytesPerPixel == 4)
        {
            format = GL_RGBA;
        }

        glGenTextures(1, &m_uiTextureId);
        glBindTexture(GL_TEXTURE_2D, m_uiTextureId);

        glTexImage2D(GL_TEXTURE_2D, 0, format, m_iWidth, m_iHeight, 0,
                     format, GL_UNSIGNED_BYTE, pSurface->pixels);
```

Load a texture file using SDL_image…

Get surface details…

Set up an OpenGL texture…

Copy the surface pixel data into the texture…

The texture.cpp Implementation…

```
        SDL_FreeSurface(pSurface);
        pSurface = 0;

        glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_NEAREST);
        glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_NEAREST);
    }
    else
    {
        LogManager::GetInstance().Log("Texture failed to load!");
        assert(0);
        return false;
    }

    return true;
}
```

Release the surface resource…

Set up nearest filtering mode

Log an error… ☺

The texture.cpp Implementation…

```
void Texture::SetActive()
{
    glBindTexture(GL_TEXTURE_2D, m_uiTextureId);
}

int Texture::GetWidth() const
{
    assert(m_iWidth);
    return (m_iWidth);
}

int Texture::GetHeight() const
{[
    assert(m_iHeight);
    return (m_iHeight);
}
```

Activate the texture on the GPU

Game Engine Design concept…

Resource management…

Game Engine: Resource Management

- What are Resources? Game-ready assets.
- Resources:
  - Assets: Textures, Sprites, Models, Animations, Sounds, Music, Movies, Level Data, etc…
  - Things that take up memory!
  - Have a location: a file path…
  - Different platforms will have different formats!

Game Engine: Resource Management

- Resource Management activities?
  - Loading… e.g. from disk to memory.
  - Unloading… no longer needed in memory… (Limited memory platforms!)
  - Using them as the game needs… (Levels, Scenes, Models, Textures, etc…)
  - Avoid loading the same resource twice!
  - Keep track of loaded resources…
  - Can be a templatized (generic) or a specific design…

The TextureManager Class

```
// COMP710 GP Framework 2025
#ifndef  ___TEXTUREMANAGER_H
#define __TEXTUREMANAGER_H

// Library Includes:
#include <string>
#include <map>

// Forward Declarations:
class Texture;

class TextureManager
{
    // Member methods:
public:
    TextureManager();
    ~TextureManager();

    bool Initialise();

    Texture* GetTexture(const char* pcFilename);

protected:

private:
    TextureManager(const TextureManager& textureManager);
    TextureManager& operator=(const TextureManager& textureManager);
```

Resource managed loading of textures from files on disk…

The TextureManager Class

```
    // Member data:
public:

protected:
    std::map<std::string, Texture*> m_pLoadedTextures;

private:

};

#endif // __TEXTUREMANAGER_H
```

An encapsulated STL container of textures…

Key: filename
Value: **Texture** pointer…

The texturemanager.cpp Implementation…

```
// COMP710 GP Framework 2025

// This include:
#include "texturemanager.h"

// Local includes:
#include "texture.h"
#include "logmanager.h"

// Library includes:
#include <cassert>
#include <SDL.h>

TextureManager::TextureManager()
{

}
```

The texturemanager.cpp Implementation…

```
TextureManager::~TextureManager()
{
    std::map<std::string, Texture*>::iterator iter = m_pLoadedTextures.begin();

    while (iter != m_pLoadedTextures.end())
    {
        Texture* pTexture = iter->second;
        delete pTexture;

        ++iter;
    }

    m_pLoadedTextures.clear();
}

bool
TextureManager::Initialise()
{
    LogManager::GetInstance().Log("TextureManager starting...");

    return true;
}
```

Release all loaded textures when destroying the **TextureManager**… Avoid memory leaks… ☺

The texturemanager.cpp Implementation…

```
Texture* TextureManager::GetTexture(const char* pcFilename)
{
    Texture* pTexture = 0;


    if (m_pLoadedTextures.find(pcFilename) == m_pLoadedTextures.end())
    {
        // Not already loaded... so load...
        pTexture = new Texture();
        if (!pTexture->Initialise(pcFilename))
        {
            LogManager::GetInstance().Log("Texture failed to initialise!");
            assert(0);
        }

        m_pLoadedTextures[pcFilename] = pTexture;
    }
    else
    {
        // It has already been loaded...
        pTexture = m_pLoadedTextures[pcFilename];
    }

    return pTexture;
}
```

Has this already been loaded?

Loading a new texture file…

Add the new texture to the resource management container…

Already loaded… simply return the **`Texture`** pointer…

Shaders…

OpenGL 3.3

and **GLSL** ☺

The Shader Class (shader.h)

```
// COMP710 GP Framework 2025
#ifndef __SHADER_H
#define __SHADER_H

#include "glew.h"

// Forward declarations:
struct Matrix4;

class Shader
{
    // Member methods:
public:
    Shader();
    ~Shader();

    bool Load(const char* vertexFile, const char* pixelFile);
    void Unload();

    void SetActive();

    void SetMatrixUniform(const char* name, const Matrix4& matrix);
    void SetVector4Uniform(const char* name, float x, float y, float z, float w);

protected:
```

Load a vertex shader and fragment shader…

Activate the shader programs on the GPU

Set uniform values on the GPU…

The Shader Class (shader.h)

```
private:
    bool IsValidProgram();

    static bool CompileShader(const char* filename, GLenum shaderType, GLuint& outShader);
    static bool IsCompiled(GLuint shader);

    // Member data:
public:

protected:
    GLuint m_vertexShader;
    GLuint m_pixelShader;
    GLuint m_shaderProgram;

private:

};

#endif // __SHADER_H
```

Helper methods for shader compilation…

OpenGL shader ID values…

The shader.cpp Implementation…

```
// COMP710 GP Framework 2025

// This include:
#include "shader.h"

// Local includes:
#include "logmanager.h"
#include "matrix4.h"

// Library includes:
#include <cassert>
#include <fstream>
#include <sstream>
#include <string>

Shader::Shader()
: m_vertexShader(0)
, m_pixelShader(0)
, m_shaderProgram(0)
{

}

Shader::~Shader()
{
    Unload();
}
```

The shader.cpp Implementation…

```
bool Shader::Load(const char* vertexFile, const char* pixelFile)
{
    bool vertexCompiled = CompileShader(vertexFile, GL_VERTEX_SHADER, m_vertexShader);
    bool pixelCompiled = CompileShader(pixelFile, GL_FRAGMENT_SHADER, m_pixelShader);


    if (vertexCompiled == false || pixelCompiled == false)
    {
        LogManager::GetInstance().Log("Shaders failed to compile!");
        assert(0);
        return false;
    }

    m_shaderProgram = glCreateProgram();

    glAttachShader(m_shaderProgram, m_vertexShader);
    glAttachShader(m_shaderProgram, m_pixelShader);
    glLinkProgram(m_shaderProgram);

    return IsValidProgram();
}

void Shader::Unload()
{
    glDeleteProgram(m_shaderProgram);
    glDeleteShader(m_vertexShader);
    glDeleteShader(m_pixelShader);
}
```

Compile the vertex shader and fragment shader code…

Attach the vertex and fragment shaders…

Clean up… deallocate shader resources…

The shader.cpp Implementation…

```
void Shader::SetActive()
{
    assert(m_shaderProgram);
    glUseProgram(m_shaderProgram);
}


void Shader::SetMatrixUniform(const char* name, const Matrix4& matrix)
{
    GLuint location = glGetUniformLocation(m_shaderProgram, name);


    glUniformMatrix4fv(location, 1, GL_TRUE, (float*)&matrix);
}

void Shader::SetVector4Uniform(const char* name, float x, float y, float z, float w)
{
    GLuint location = glGetUniformLocation(m_shaderProgram, name);

    float vec4[4];

    vec4[0] = x;
    vec4[1] = y;
    vec4[2] = z;
    vec4[3] = w;

    glUniform4fv(location, 1, vec4);
}
```

Activate the shader programs on the GPU

Set uniform values on the GPU…

The shader.cpp Implementation…

```
bool Shader::CompileShader(const char* filename, GLenum shaderType, GLuint& outShader)
{
  std::ifstream shaderFile(filename);

  if (shaderFile.is_open())
  {
      std::stringstream sstream;
      sstream << shaderFile.rdbuf();
      std::string shaderCode = sstream.str();
      const char* pShaderCode = shaderCode.c_str();

      outShader = glCreateShader(shaderType);
      glShaderSource(outShader, 1, &(pShaderCode), 0);
      glCompileShader(outShader);

      if (!IsCompiled(outShader))
      {
          LogManager::GetInstance().Log("Shader failed to compile!");
          return false;
      }
  }
  else
  {
      LogManager::GetInstance().Log("Shader file not found!");
      return false;
  }

  return true;
}
```

Read in the shader source code…

Compile the shader source code…

The shader.cpp Implementation…

```
bool Shader::IsCompiled(GLuint shader)
{
    GLint compiledStatus;

    glGetShaderiv(shader, GL_COMPILE_STATUS, &compiledStatus);

    if (compiledStatus != GL_TRUE)
    {
        char error[1024];
        error[0] = 0;
        glGetShaderInfoLog(shader, 1023, 0, error);

        LogManager::GetInstance().Log("Shader failed to compile!");

        return false;
    }

    return true;
}
```

Check the shader source code compiled…

The shader.cpp Implementation…

```
bool Shader::IsValidProgram()
{
    GLint linkedStatus;

    glGetProgramiv(m_shaderProgram, GL_LINK_STATUS, &linkedStatus);

    if (linkedStatus != GL_TRUE)
    {
        char error[1024];
        error[0] = 0;
        glGetShaderInfoLog(m_shaderProgram, 1023, 0, error);

        LogManager::GetInstance().Log("Shader failed to link!");
        assert(0);

        return false;
    }

    return true;
}
```

Check the shader program was created…

Add “shaders” assets folder…

To the **game** folder…

The `sprite.vert` File…

```
#version 330

uniform mat4 uWorldTransform;
uniform mat4 uViewProj;

layout(location = 0) in vec3 inPosition;
layout(location = 1) in vec2 inTexCoord;

out vec2 fragTexCoord;

void main()
{
    vec4 pos = vec4(inPosition, 1.0);

    gl_Position = pos * uWorldTransform * uViewProj;

    fragTexCoord = inTexCoord;
}
```

C++ will input a world transform matrix, and a view/projection matrix to this shader…

This vertex shader operators on vertices that have three **`float`** values (x, y, z) position, and then two **`float`** values (u, v) coordinates…

Outputs a texture coordinate for the fragment shader to then operate on…

Compute the vertex position using the GPU…

- Vertex shaders:
  - Processes individual vertices…
  - Programmable graphics pipeline…

The `sprite.frag` File…

```
#version 330

in vec2 fragTexCoord;

out vec4 outColor;

uniform sampler2D uTexture;
uniform vec4 colour;

void main()
{
     outColor = colour * texture(uTexture, fragTexCoord);
}
```

Takes input from the output of the vertex shader…

Outputs a colour value…

The C++ program will set a texture uniform, and a colour value… (the tint and alpha ☺)

Compute the colour of this fragment using the GPU…

- Fragment shaders:
  - Processes individual fragments ("a pixel")…
    - Microsoft's DirectX Graphics API calls this a "Pixel Shader"
  - Programmable graphics pipeline…

Back to C++ code…

A basic 4x4 matrix structure… ☺

The matrix4.h File…

```
// COMP710 GP Framework 2025
#ifndef __MATRIX4_H
#define __MATRIX4_H

struct Matrix4
{
    float m[4][4];
};

void SetZero(Matrix4& mat);
void SetIdentity(Matrix4& mat);
void CreateOrthoProjection(Matrix4& mat, float width, float height);

#endif // __MATRIX4_H
```

A 4-by-4 array…

The matrix4.cpp File…

```
// COMP710 GP Framework 2022
#include "matrix4.h"

void SetZero(Matrix4& mat)
{
    mat.m[0][0] = 0.0f;
    mat.m[0][1] = 0.0f;
    mat.m[0][2] = 0.0f;
    mat.m[0][3] = 0.0f;

    mat.m[1][0] = 0.0f;
    mat.m[1][1] = 0.0f;
    mat.m[1][2] = 0.0f;
    mat.m[1][3] = 0.0f;

    mat.m[2][0] = 0.0f;
    mat.m[2][1] = 0.0f;
    mat.m[2][2] = 0.0f;
    mat.m[2][3] = 0.0f;

    mat.m[3][0] = 0.0f;
    mat.m[3][1] = 0.0f;
    mat.m[3][2] = 0.0f;
    mat.m[3][3] = 0.0f;
}
```

Set all elements of the matrix to the value zero…

The matrix4.cpp File…

```
void SetIdentity(Matrix4& mat)
{
    SetZero(mat);

    mat.m[0][0] = 1.0f;
    mat.m[1][1] = 1.0f;
    mat.m[2][2] = 1.0f;
    mat.m[3][3] = 1.0f;
}

void CreateOrthoProjection(Matrix4& mat, float width, float height)
{
    SetZero(mat);

    mat.m[0][0] = 2.0f / width;
    mat.m[1][1] = 2.0f / -height;
    mat.m[2][2] = -2.0f / (1.0f - -1.0f);

    mat.m[3][0] = -1.0f;
    mat.m[3][1] = 1.0f;
    mat.m[3][2] = 0.0f;
    mat.m[3][3] = 1.0f;
}
```

Set the matrix to the identity…

Generate a basic 2D orthographic projection matrix…

The VertexArray Class

```
// COMP710 GP Framework 2025
#ifndef __VERTEXARRAY_H
#define __VERTEXARRAY_H

class VertexArray
{
    // Member methods:
public:
    VertexArray(const float* pVertices, unsigned int numVertices,
                const unsigned int* pIndices, unsigned int numIndicies);
    ~VertexArray();

    void SetActive();

    unsigned int GetNumVertices() const;
    unsigned int GetNumIndicies() const;

protected:

private:
    VertexArray(const VertexArray& vertexArray);
    VertexArray& operator=(const VertexArray& vertexArray);

    // Member data:
public:

protected:
    unsigned int m_numVertices;
```

The VertexArray Class

```
    unsigned int m_numIndices;

    unsigned int m_glVertexBuffer;
    unsigned int m_glIndexBuffer;
    unsigned int m_glVertexArray;

private:

};

#endif // __VERTEXARRAY_H
```

The vertexarray.cpp Implementation…

```
// COMP710 GP Framework 2025

// This include:
#include "vertexarray.h"

// Local includes:

// Library includes:
#include <glew.h>
#include <cassert>

VertexArray::VertexArray(const float* pVertexData, unsigned int numVertices,
                         const unsigned int* pIndexData, unsigned int numIndicies)
: m_numVertices(numVertices)
, m_numIndices(numIndicies)
, m_glVertexBuffer(0)
, m_glIndexBuffer(0)
, m_glVertexArray(0)
{
    const int stride = 5 * sizeof(float); //XYZUV

    assert(pVertexData);

    glGenVertexArrays(1, &m_glVertexArray);
    glBindVertexArray(m_glVertexArray);

    glGenBuffers(1, &m_glVertexBuffer);
    glBindBuffer(GL_ARRAY_BUFFER, m_glVertexBuffer);
    glBufferData(GL_ARRAY_BUFFER, m_numVertices * stride, pVertexData, GL_STATIC_DRAW);
```

Set up vertex and index data on the GPU…

The vertexarray.cpp Implementation…

```
    if (pIndexData)
    {
        glGenBuffers(1, &m_glIndexBuffer);
        glBindBuffer(GL_ELEMENT_ARRAY_BUFFER, m_glIndexBuffer);
        glBufferData(GL_ELEMENT_ARRAY_BUFFER, m_numIndices * sizeof(unsigned int), pIndexData, GL_STATIC_DRAW);
    }

    // Layout: XYZ
    glEnableVertexAttribArray(0);
    glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, stride, 0);

    // Layout: UV
    glEnableVertexAttribArray(1);
    glVertexAttribPointer(1, 2, GL_FLOAT, GL_FALSE, stride, reinterpret_cast<void*>(sizeof(float) * 3));
}

VertexArray::~VertexArray()
{
    glDeleteBuffers(1, &m_glVertexBuffer);
    glDeleteBuffers(1, &m_glIndexBuffer);
    glDeleteVertexArrays(1, &m_glVertexArray);
}
```

Configure the vertex layout on the GPU…

Deallocated resources…

The `vertexarray.cpp` Implementation…

```
void
VertexArray::SetActive()
{
    glBindVertexArray(m_glVertexArray);
}

unsigned int
VertexArray::GetNumVertices() const
{
    return m_numVertices;
}

unsigned int
VertexArray::GetNumIndicies() const
{
    return m_numIndices;
}
```

Activate the vertex data on the GPU… so we can render with it… ☺

Almost done…

Let’s create a **`Sprite`** class… ☺

The Sprite Class (sprite.h)

```
// COMP710 GP Framework 2025
#ifndef __SPRITE_H
#define __SPRITE_H

// Forward Declarations:
class Renderer;
class Texture;

class Sprite
{
    // Member methods:
public:
    Sprite();
    ~Sprite();

    bool Initialise(Texture& texture);
    void Process(float deltaTime);
    void Draw(Renderer& renderer);

    int GetWidth() const;
    int GetHeight() const;

    void SetX(int x);
    int GetX() const;
    void SetY(int y);
    int GetY() const;

    void SetAngle(float angle);
    float GetAngle() const;
```

The Sprite Class (sprite.h)

```
    void SetScale(float scale);
    float GetScale() const;

    void SetAlpha(float alpha);
    float GetAlpha() const;

    void SetRedTint(float value);
    float GetRedTint() const;
    void SetGreenTint(float value);
    float GetGreenTint() const;
    void SetBlueTint(float value);
    float GetBlueTint() const;

protected:
    float Clamp(float minimum, float value, float maximum);

private:
    Sprite(const Sprite& sprite);
    Sprite& operator=(const Sprite& sprite);

    // Member data:
public:

protected:
    Texture* m_pTexture;
    int m_x;
    int m_y;
```

The Sprite Class (sprite.h)

```
    float m_angle;
    int m_centerX;
    int m_centerY;

    int m_width;
    int m_height;

    float m_scale;
    float m_alpha;

    float m_tintRed;
    float m_tintGreen;
    float m_tintBlue;

private:

};

#endif // __SPRITE_H
```

The sprite.cpp Implementation…

```
// COMP710 GP Framework 2025

// This include:
#include "sprite.h"

// Local includes:
#include "renderer.h"
#include "texture.h"

Sprite::Sprite()
: m_pTexture(0)
, m_x(0)
, m_y(0)
, m_width(0)
, m_height(0)
, m_angle(0.0f)
, m_centerX(0)
, m_centerY(0)
, m_scale(1.0f)
, m_alpha(1.0f)
, m_tintRed(1.0f)
, m_tintGreen(1.0f)
, m_tintBlue(1.0f)
{

}
```

The sprite.cpp Implementation…

```
Sprite::~Sprite()
{

}

bool
Sprite::Initialise(Texture& texture)
{
    m_pTexture = &texture;

    m_width = m_pTexture->GetWidth();
    m_height = m_pTexture->GetHeight();

    return true;
}

void
Sprite::Process(float deltaTime)
{

}

void
Sprite::Draw(Renderer& renderer)
{
    m_pTexture->SetActive();
    renderer.DrawSprite(*this);
}
```

The sprite.cpp Implementation…

```
void
Sprite::SetX(int x)
{
    m_x = x;
}

void
Sprite::SetY(int y)
{
    m_y = y;
}

int
Sprite::GetX() const
{
    return m_x;
}

int
Sprite::GetY() const
{
    return m_y;
}
```

The sprite.cpp Implementation…

```
void
Sprite::SetAngle(float angle)
{
    while (angle > 360)
    {
        angle -= 360;
    }

    m_angle = angle;
}

float
Sprite::GetAngle() const
{
    return m_angle;
}

void
Sprite::SetScale(float scale)
{
    m_scale = scale;
}

float
Sprite::GetScale() const
{
    return m_scale;
}
```

The sprite.cpp Implementation…

```
void
Sprite::SetAlpha(float alpha)
{
    m_alpha = Clamp(0.0f, alpha, 1.0f);
}

float
Sprite::GetAlpha() const
{
    return m_alpha;
}

int
Sprite::GetWidth() const
{
    return static_cast<int>(ceilf(m_width * m_scale));
}

int
Sprite::GetHeight() const
{
    return static_cast<int>(ceilf(m_height * m_scale));
}
```

The sprite.cpp Implementation…

```
float
Sprite::Clamp(float minimum, float value, float maximum)
{
    if (value > maximum)
    {
       value = maximum;
    }
    else if (value < minimum)
    {
        value = minimum;
    }

    return value;
}

void
Sprite::SetRedTint(float value)
{
    m_tintRed = Clamp(0.0f, value, 1.0f);
}

float
Sprite::GetRedTint() const
{
    return m_tintRed;
}
```

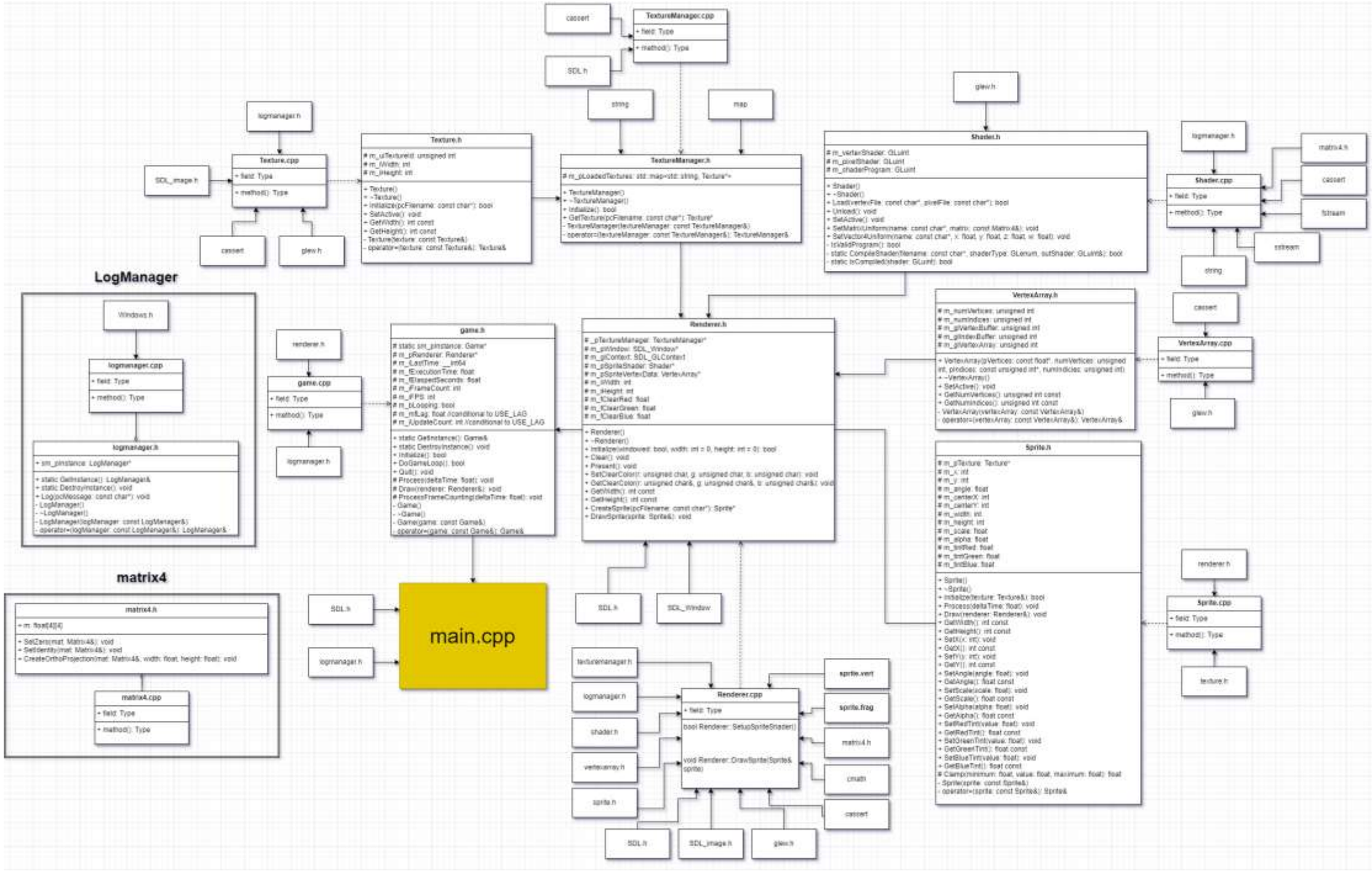
The sprite.cpp Implementation…

```
void
Sprite::SetGreenTint(float value)
{
    m_tintGreen = Clamp(0.0f, value, 1.0f);
}

float
Sprite::GetGreenTint() const
{
    return m_tintGreen;
}

void
Sprite::SetBlueTint(float value)
{
    m_tintBlue = Clamp(0.0f, value, 1.0f);
}

float
Sprite::GetBlueTint() const
{
    return m_tintBlue;
}
```

Okay… let’s compile and see what happens…

Do you have any errors or warnings? ☺

“Build” the project first! Don’t just run! Check the **Output window** for compiler warnings and errors, and linker errors! ☺


cassert
TextureManager.cpp
+ field: Type
+ method(): Type
SDL.h
string
map
glew.h
logmanager.h
Texture.cpp
+ field: Type
+ method(): Type
SDL_image.h
cassert
glew.h
Texture.h
m_uiTextureId: unsigned int
m_iWidth: int
m_iHeight: int
+ Texture()
+ ~Texture()
+ Initialize(pcFilename: const char*): bool
+ SetActive(): void
+ GetWidth(): int const
+ GetHeight(): int const
- Texture(texture: const Texture&)
- operator=(texture: const Texture&): Texture&
TextureManager.h
m_pLoadedTextures: std::map<std::string, Texture*>
+ TextureManager()
+ ~TextureManager()
+ Initialize(): bool
+ GetTexture(pcFilename: const char*): Texture*
- TextureManager(textureManager: const TextureManager&)
- operator=(textureManager: const TextureManager&): TextureManager&
Shader.h
m_vertexShader: GLuint
m_pixelShader: GLuint
m_shaderProgram: GLuint
+ Shader()
+ ~Shader()
+ Load(vertexFile: const char*, pixelFile: const char*): bool
+ Unload(): void
+ SetActive(): void
+ SetMatrixUniform(name: const char*, matrix: const Matrix4&): void
+ SetVector4Uniform(name: const char*, x: float, y: float, z: float, w: float): void
- IsValidProgram(): bool
- static CompileShader(filename: const char*, shaderType: GLenum, outShader: GLuint&): bool
- static IsCompiled(shader: GLuint): bool
logmanager.h
matrix4.h
Shader.cpp
+ field: Type
+ method(): Type
cassert
fstream
sstream
string
cassert
VertexArray.cpp
+ field: Type
+ method(): Type
glew.h
LogManager
Windows.h
logmanager.cpp
+ field: Type
+ method(): Type
logmanager.h
+ sm_pInstance: LogManager*
+ static GetInstance(): LogManager&
+ static DestroyInstance(): void
+ Log(pcMessage: const char*): void
- LogManager()
- ~LogManager()
- LogManager(logManager: const LogManager&)
- operator=(logManager: const LogManager&): LogManager&
renderer.h
game.cpp
+ field: Type
+ method(): Type
logmanager.h
game.h
static sm_pInstance: Game*
m_pRenderer: Renderer*
m_LastTime: __int64
m_fExecutionTime: float
m_fElapsedSeconds: float
m_iFrameCount: int
m_iFPS: int
m_bLooping: bool
m_mLag: float //conditional to USE_LAG
m_iUpdateCount: int //conditional to USE_LAG
+ static GetInstance(): Game&
+ static DestroyInstance(): void
+ Initialize(): bool
+ DoGameLoop(): bool
+ Quit(): void
# ProcessdeltaTime: float): void
# Draw(renderer: Renderer&): void
# ProcessFrameCounting(deltaTime: float): void
- Game()
- ~Game()
- Game(game: const Game&)
- operator=(game: const Game&): Game&
Renderer.h
_pTextureManager: TextureManager*
m_pWindow: SDL_Window*
m_glContext: SDL_GLContext
m_pSpriteShader: Shader*
m_pSpriteVertexData: VertexArray*
m_iWidth: int
m_iHeight: int
m_fClearRed: float
m_fClearGreen: float
m_fClearBlue: float
+ Renderer()
+ ~Renderer()
+ Initialize(windowed: bool, width: int = 0, height: int = 0): bool
+ Clear(): void
+ Present(): void
+ SetClearColor(r: unsigned char, g: unsigned char, b: unsigned char): void
+ GetClearColor(r: unsigned char&, g: unsigned char&, b: unsigned char&): void
+ GetWidth(): int const
+ GetHeight(): int const
+ CreateSprite(pcFilename: const char*): Sprite*
+ DrawSprite(sprite: Sprite&): void
VertexArray.h
m_numVertices: unsigned int
m_numIndices: unsigned int
m_glVertexBuffer: unsigned int
m_glIndexBuffer: unsigned int
m_glVertexArray: unsigned int
+ VertexArray(pVertices: const float*, numVertices: unsigned int, pIndices: const unsigned int*, numIndicies: unsigned int)
+ ~VertexArray()
+ SetActive(): void
+ GetNumVertices(): unsigned int const
+ GetNumIndices(): unsigned int const
- VertexArray(vertexArray: const VertexArray&)
- operator=(vertexArray: const VertexArray&): VertexArray&
Sprite.h
m_pTexture: Texture*
m_x: int
m_y: int
m_angle: float
m_centerX: int
m_centerY: int
m_width: int
m_height: int
m_scale: float
m_alpha: float
m_tintRed: float
m_tintGreen: float
m_tintBlue: float
+ Sprite()
+ ~Sprite()
+ Initialize(texture: Texture&): bool
+ Process(deltaTime: float): void
+ Draw(renderer: Renderer&): void
+ GetWidth(): int const
+ GetHeight(): int const
+ SetX(x: int): void
+ GetX(): int const
+ SetY(): int: void
+ GetY(): int const
+ SetAngle(angle: float): void
+ GetAngle(): float const
+ SetScale(scale: float): void
+ GetScale(): float const
+ SetAlpha(alpha: float): void
+ GetAlpha(): float const
+ SetRedTint(value: float): void
+ GetRedTint(): float const
+ SetGreenTint(value: float): void
+ GetGreenTint(): float const
+ SetBlueTint(value: float): void
+ GetBlueTint(): float const
# Clamp(minimum: float, value: float, maximum: float): float
- Sprite(sprite: const Sprite&)
- operator=(sprite: const Sprite&): Sprite&
renderer.h
Sprite.cpp
+ field: Type
+ method(): Type
texture.h
matrix4
matrix4.h
+ m: float[4][4]
+ SetZero(mat: Matrix4&): void
+ SetIdentity(mat: Matrix4&): void
+ CreateOrthoProjection(mat: Matrix4&, width: float, height: float): void
matrix4.cpp
+ field: Type
+ method(): Type
SDL.h
logmanager.h
main.cpp
SDL.h
SDL_Window
texturemanager.h
logmanager.h
shader.h
vertexarray.h
sprite.h
Renderer.cpp
+ field: Type
bool Renderer::SetupSpriteShader()
void Renderer::DrawSprite(Sprite& sprite)
sprite.vert
sprite.frag
matrix4.h
cmath
cassert
SDL.h
SDL_image.h
glew.h

Beware, we have no input system yet… ☺

For today… run the “GP Framework” from within the IDE, so you can press the **STOP** button to exit the process…

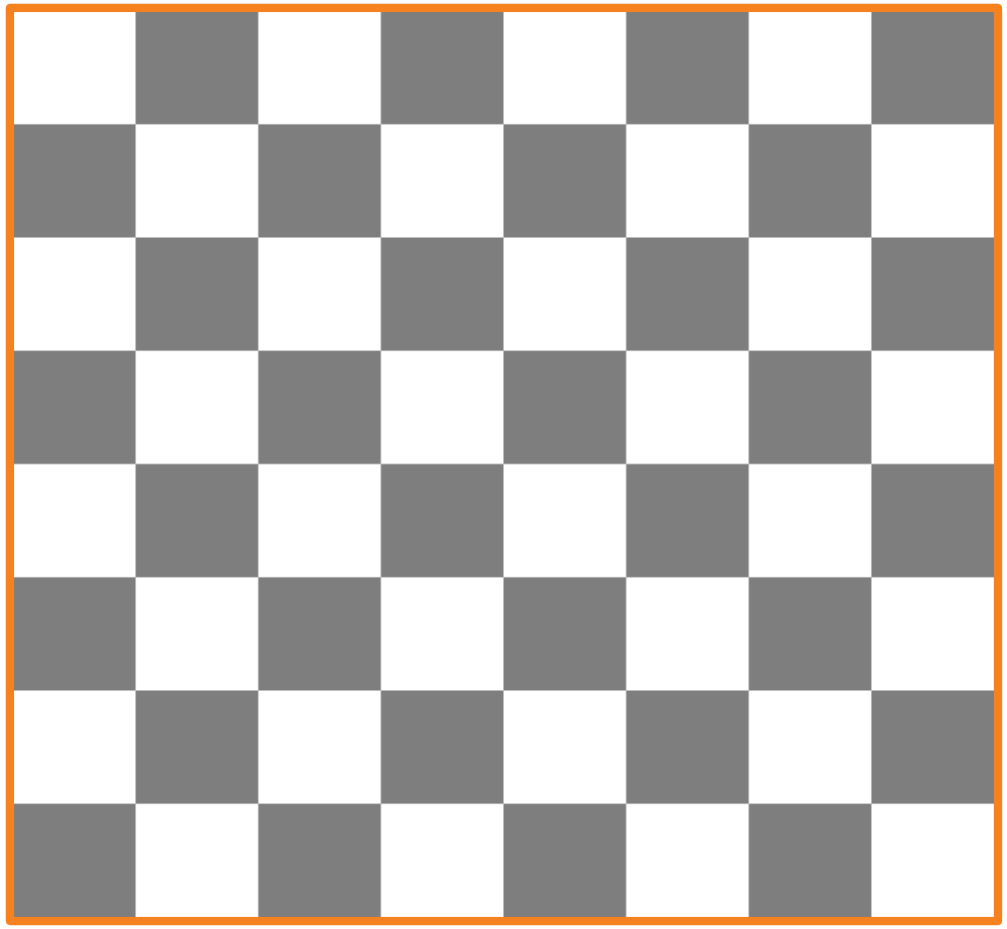
Let’s load a sprite…

Here’s are some PNG files to use…

Shared PNG files…

- Go to:
  `Studio5Resources\Sprites\`
  - **`ball.png`**
  - **`board8x1.png`**
  - **`board8x2.png`**
  - **`board8x8.png`**

The ball has alpha transparency at the corners… ☺



Copy the shared assets to your `\assets\` folder

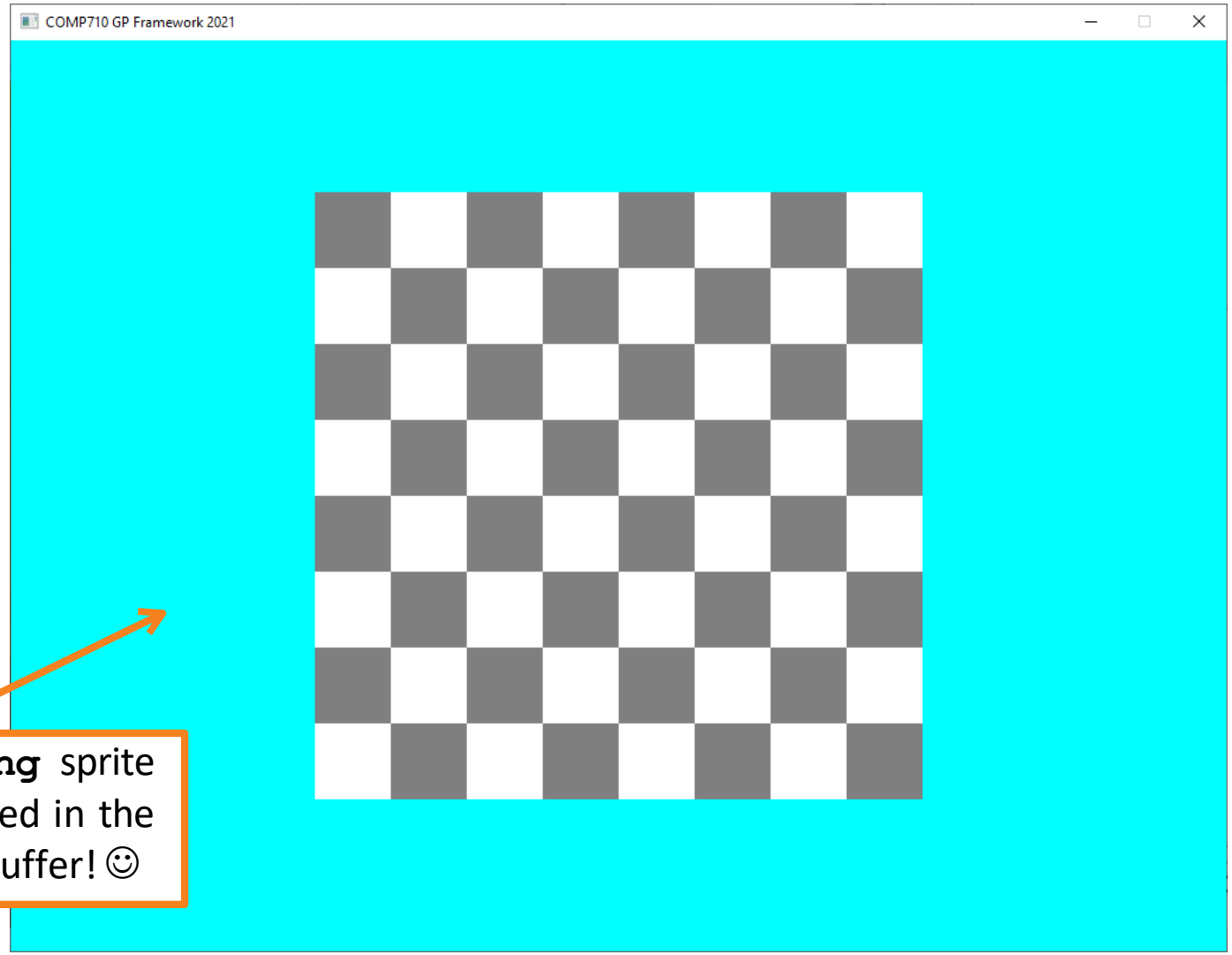
`ball.png`, **`board8x1.png`**, **`board8x2.png`** and **`board8x8.png`**

To Load and Draw a Sprite…

- In **`game.h`** add a **`Sprite*`** member named **`m_pCheckerboard`**.
  - And… add a forward declaration for **`Sprite`** to **`game.h`**
- In **`Game::Initialise()`**
  - Call **`m_pRenderer->CreateSprite()`**
    - Load the **`board8x8.png`** file… (and don't forget to **`#include "sprite.h"`** ☺)
    - Save the **`Sprite*`** returned from this call into the **`m_pCheckerboard`** member…
    - Set the X and Y coordinate positions of the sprite…
- In **`Game::Process()`**
  - Call the member sprite's Process method…
- In **`Game::Draw()`**
  - Call the member sprites Draw method…

Also… try changing the backbuffer clear colour in the game's initialise method too… ☺

To Load and Draw a Sprite…



The **`board8x8.png`** sprite successfully rendered in the centre of the backbuffer! ☺

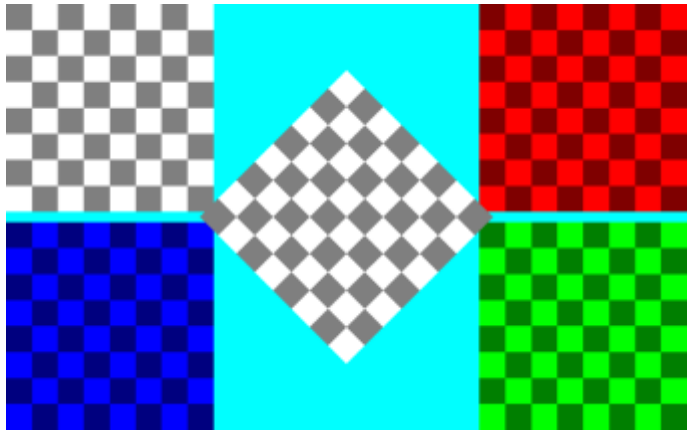
Again, we have no input system yet… ☺

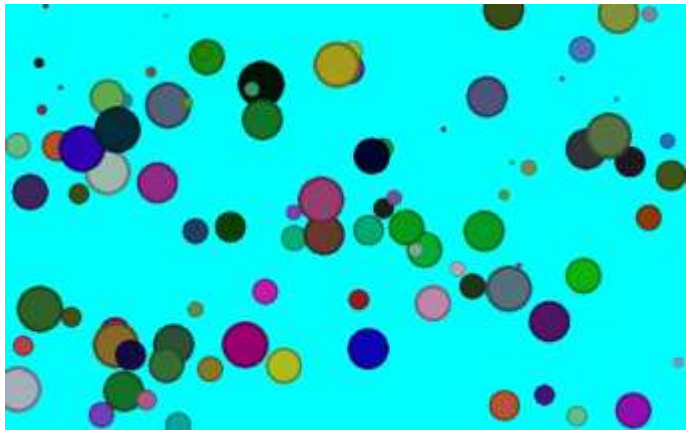
For today… run the “GP Framework” from within the IDE, so you can press the **STOP** button to exit the process…

We will implement this (and more!) in the next Studio Session… ☺

Exercises: Studio Session 05 (SS05)

- Learn more sprite rendering by completing a small “SS05” exercise:
  - C++ – Using Sprites
  - C++ – Bouncing Balls





- Add the completed exercises to your Personal Portfolio – and reflect on what you have learnt! ☺

Summary

- Video Game Structure
- Real-Time Simulation: The Game Loop, Computer Graphics…
- Game Engine Concepts:
  - Resource Management
  - Build Targets
- The COMP710 “GP Framework”:
  - Project Configuration… Build Targets… Project File Structure…
  - Logging, The **`Game`** Object, The **`main`** Function, The **`Renderer`** Interface
  - **`Texture`**, **`TextureManager`**, **`Shader`**, GLSL, **`Matrix4`**, **`VertexArray`**
  - Sprites!